

FINAL LAB: Servo Motor final project

Name: Tony Han

Table partners: Pranay, Marnu

#1. Experiment description and Prelab

##1.1 Description

This lab is the final lab; I will be building the actual servo motor control. There is no pre-lab. Rather, I will document each of my steps in building this circuit.

I will follow the below diagram to build my circuit; I will also illustrate my own circuit diagram and include photos of my circuit.

In [3]: `InlineImage('servo.png', width=12, height=12, caption= "Figure 1, prelab ser`

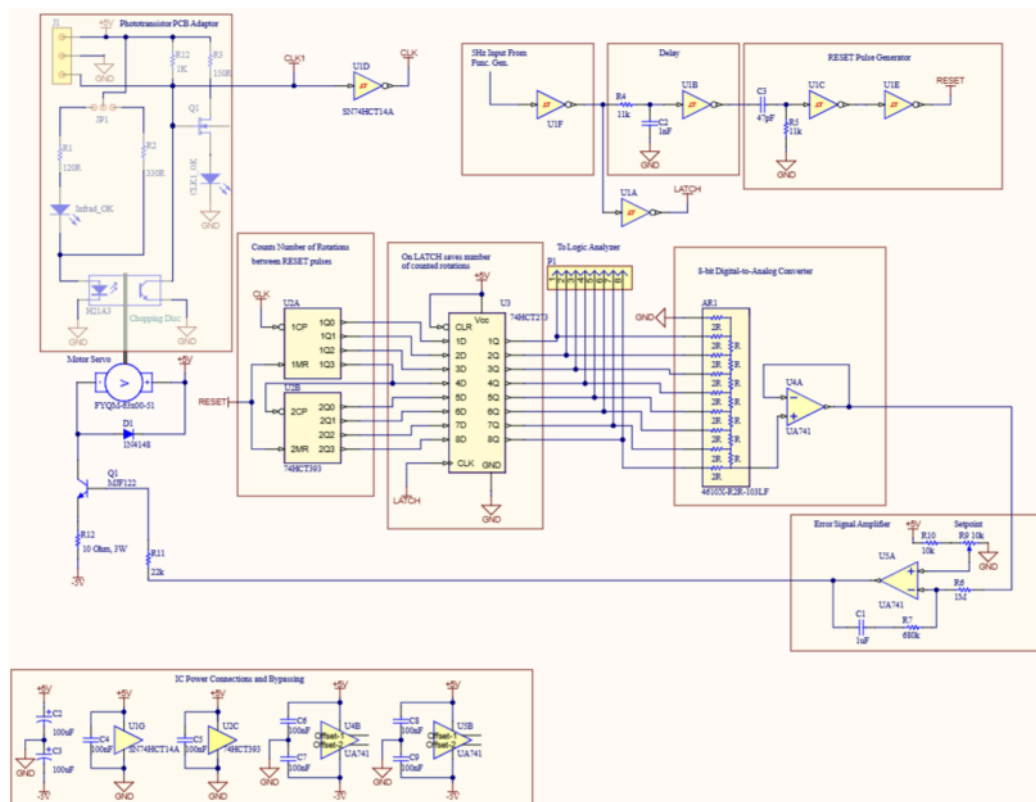


Figure 1: This is the entire setup of the servo control circuit. The lower panel shows the power connections for each IC used.

Figure 1, prelab servo diagram.

#2. Experiment tasks

##2.1 Task 1, Delay and reset generator

For the first task, I will build the delay and reset generator according to the lab manual.

I will use:

- a 74HC14A schmitt trigger inverter
- 2 11K resistors
- 1 1nF capacitor
- 1 47pF capacitor

Here is an image of the portion of the circuit. Each individual subsection will have its own images/diagrams for clarity. This is for general reference for the overall portion.

```
In [7]: #image here
InlineImage('partone.png', width=12, height=12, caption= "Figure 2, delay ar
```

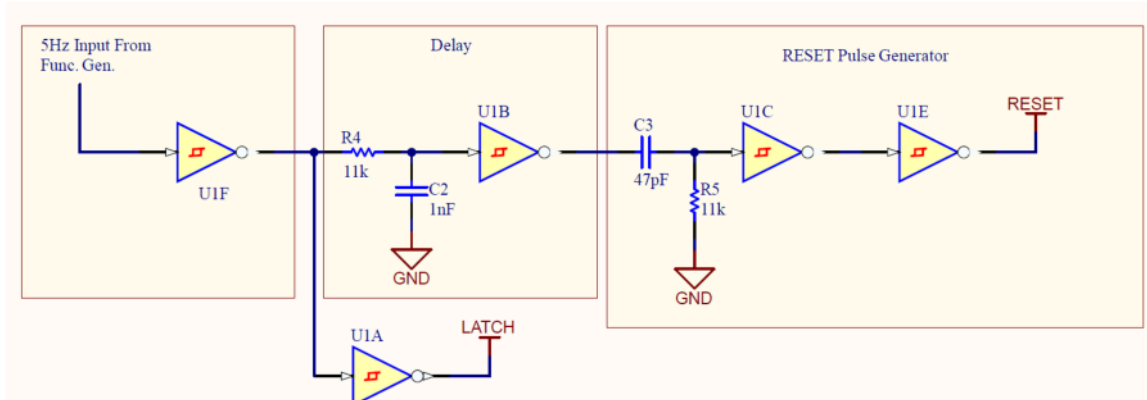


Figure 2, delay and reset pulse generator.

Here is the datasheet for the 74HC14A:

```
In [9]: InlineImage('latchsheet.png', width=12, height=12, caption= "Figure 3, 74HC1
InlineImage('latchsheet2.png', width=12, height=12, caption= "Figure 4, 74HC
```

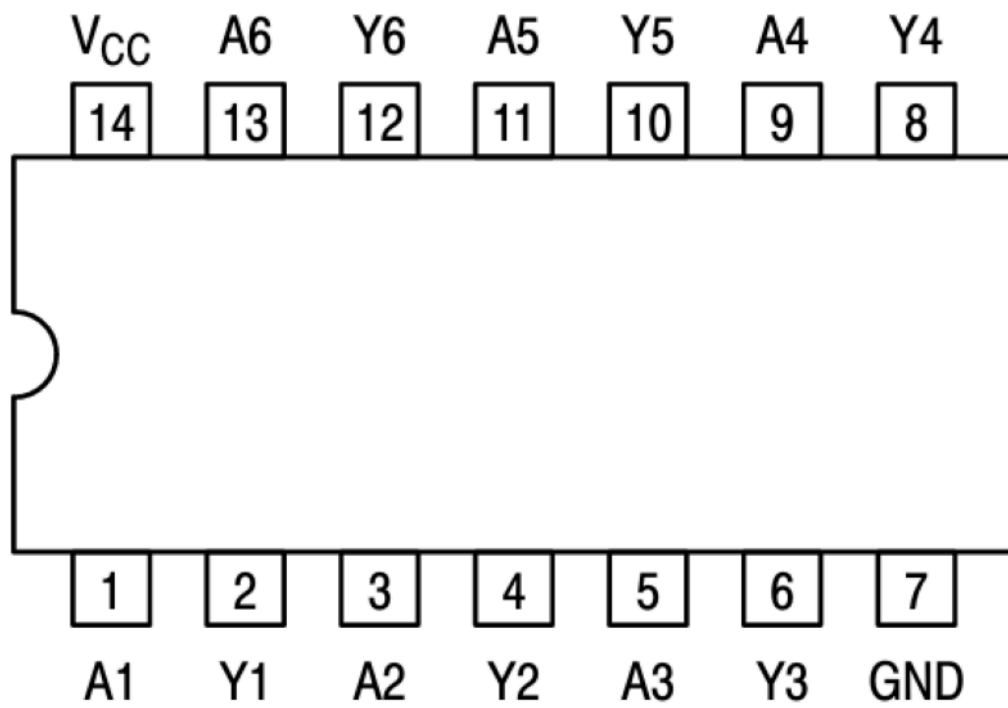


Figure 3, 74HC14A pin diagram

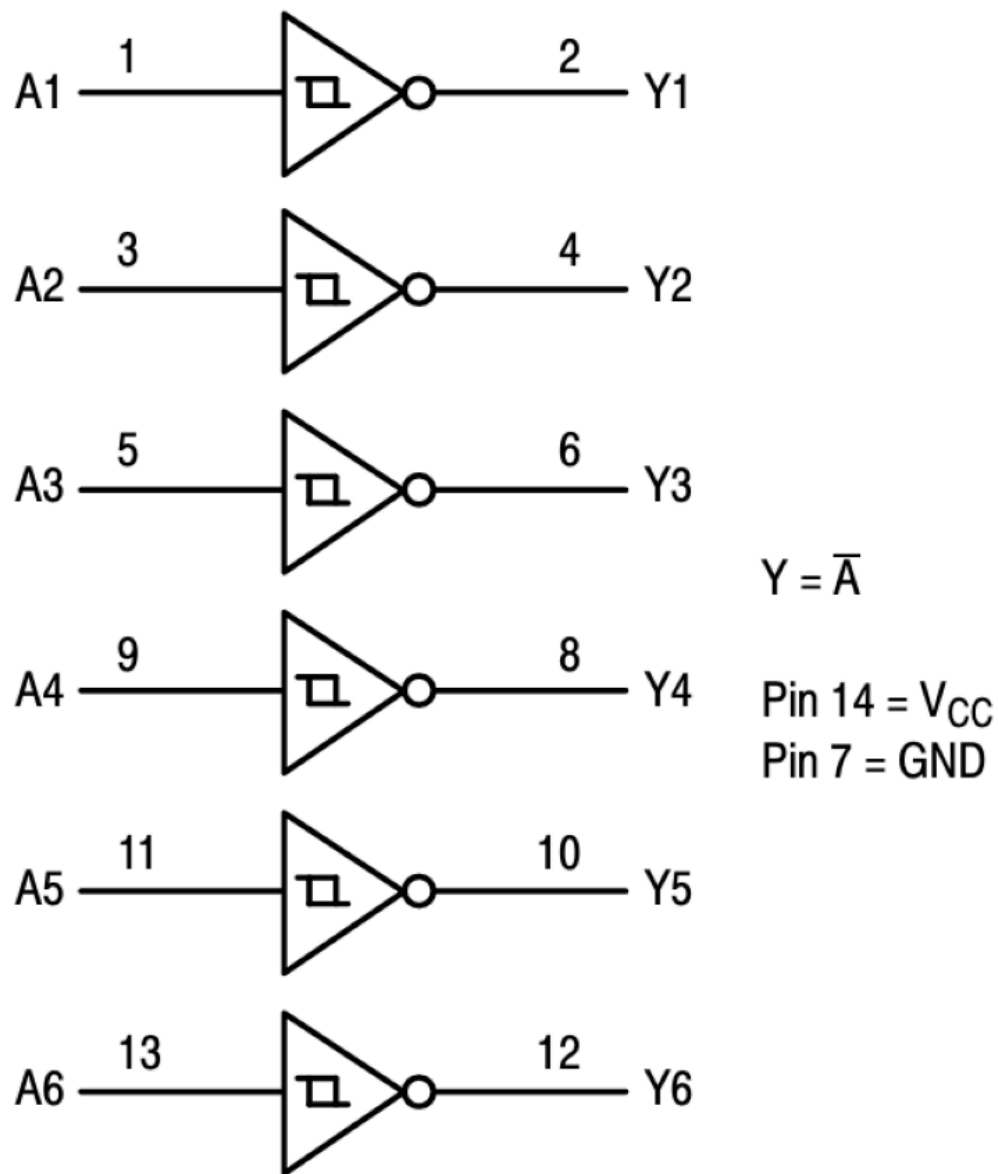


Figure 4, 74HC14A logic diagram

Here is a circuit diagram, and here is a photo of my circuit. I have built the latch, the delay, and the reset pulse generator in one go for efficiency. However, for each particular section, I will include a schematic for that single part of the circuit for clarity. Here, I will include a photo and circuit diagram of the full circuit:

```
In [11]: #circuit diagram
         InlineImage('firstpart.png', width=12, height=12, caption= "Figure 5, Circui
```

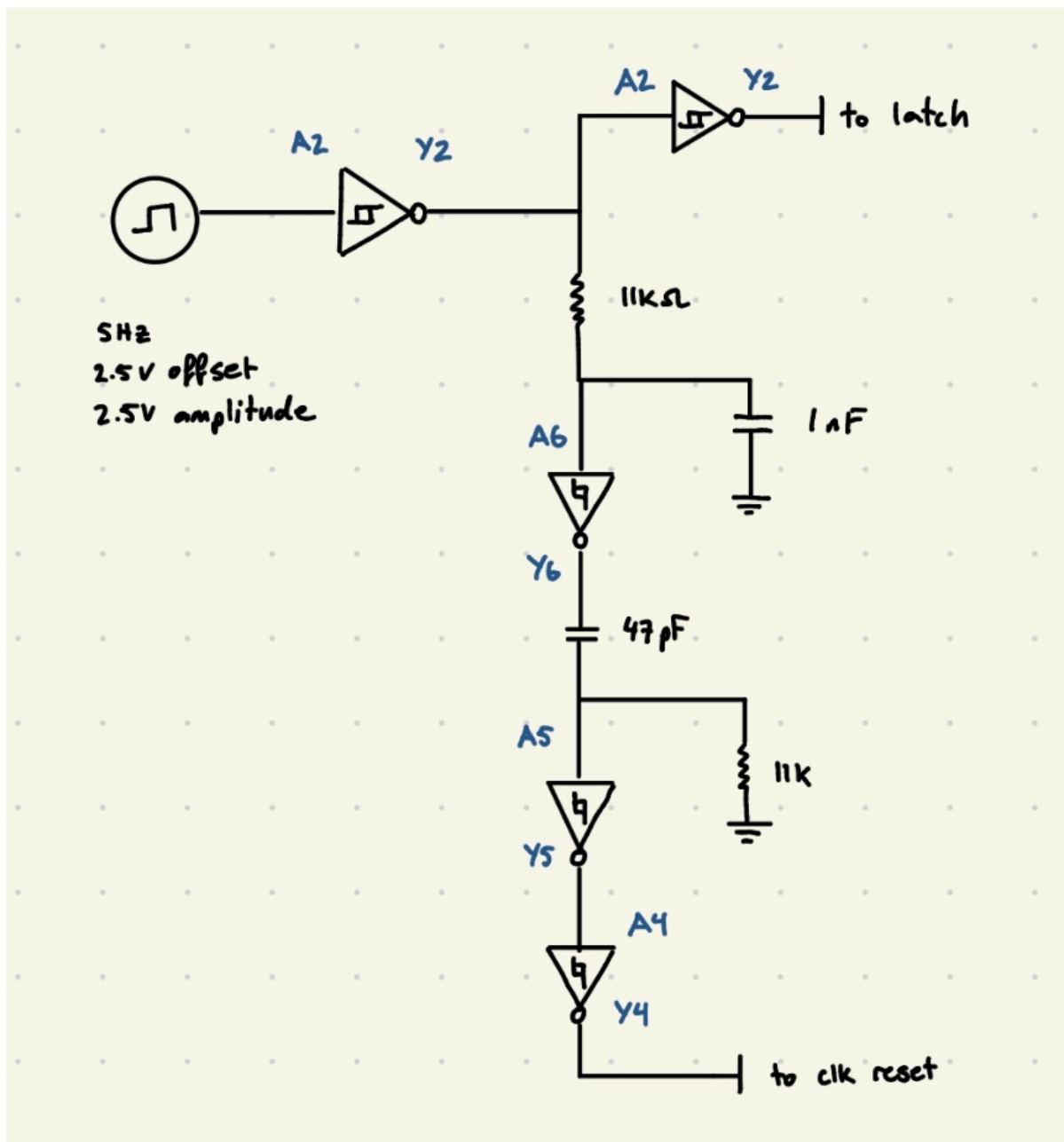



Figure 5, Circuit schematic for my delay and reset pulse generator.

In [91]: `#up to fig 5`
`InlineImage('updatedfirst.png', width=12, height=12, caption= "Figure 6, pho`

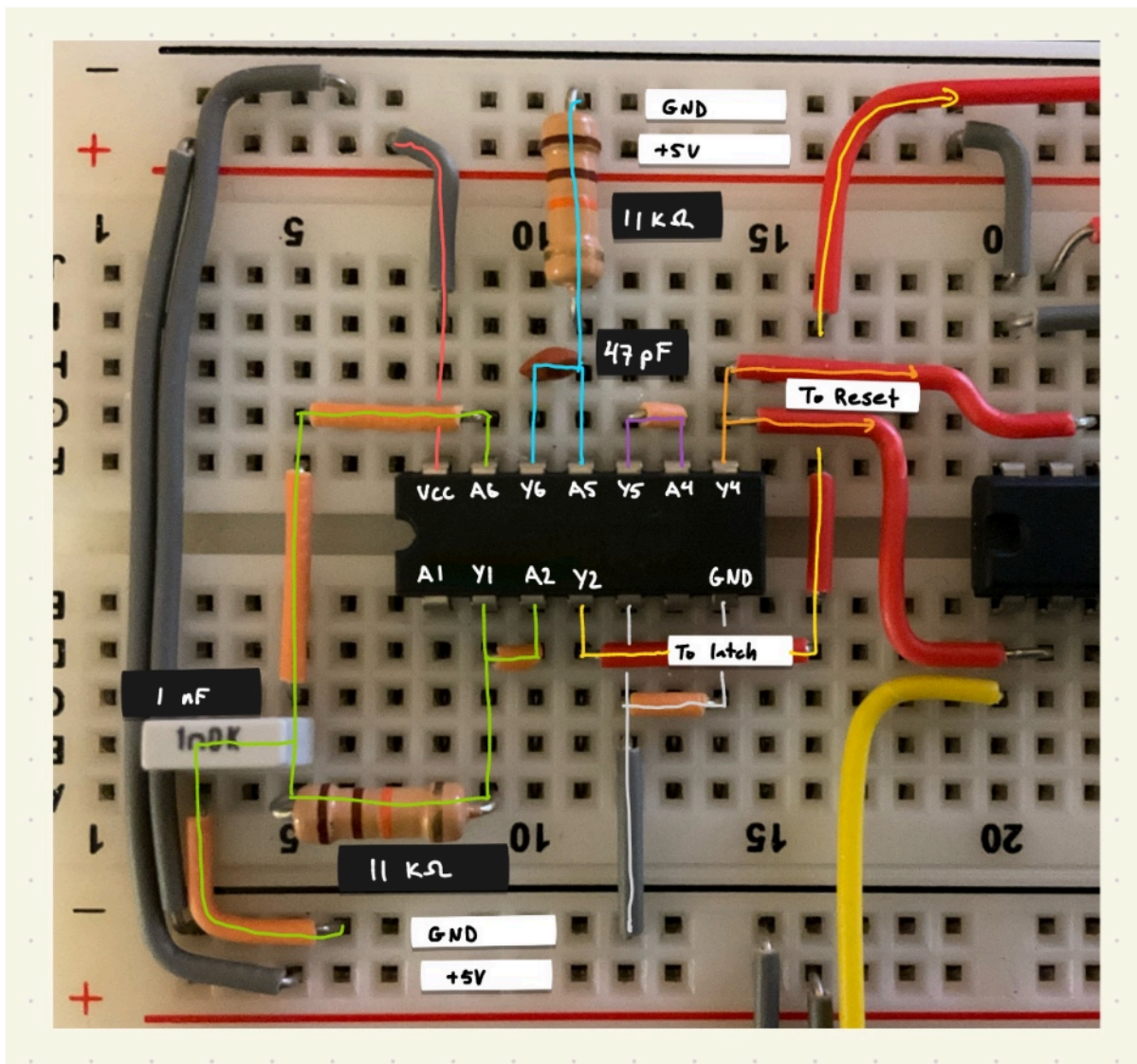


Figure 6, photo of my delay and reset circuit.

Here is what I see in my scope when probing the input square wave (orange) and the output on latch (blue):

In [13]: `InlineImage('latchwaveform.png', width=12, height=12, caption= "Figure 7, wa`

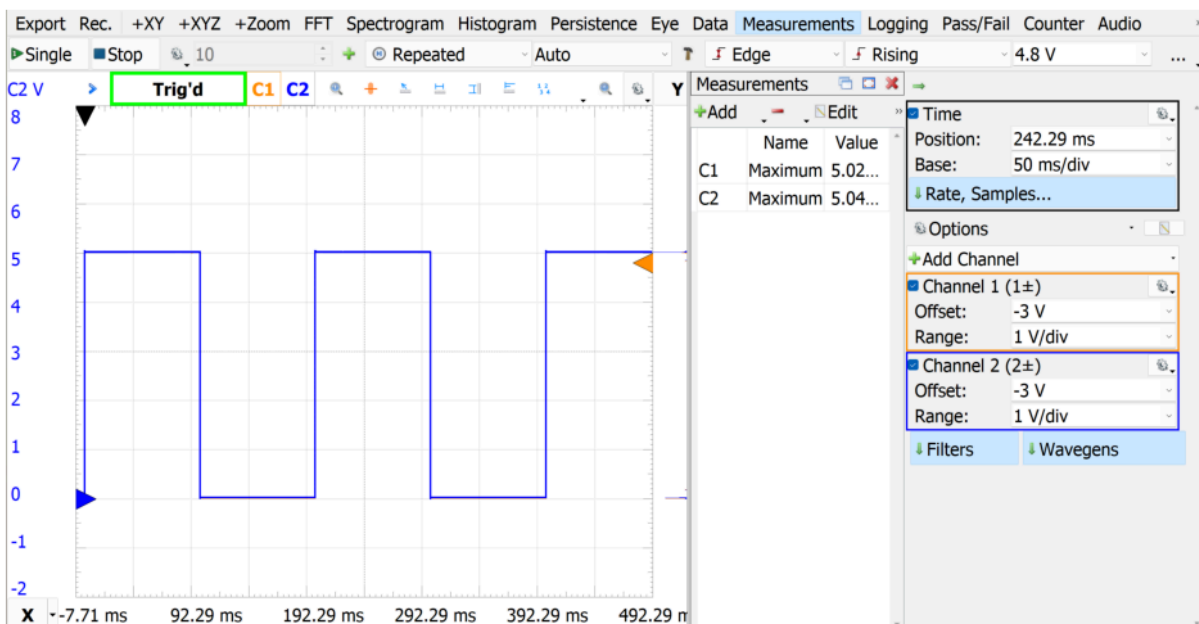


NOTE: the waveforms are directly superimposed upon each other; the blue is on top, but you can see the orange below it if you look closely.

TS This has the general expected shape; the latch should match the input. However, I notice some roughness on the latch output at the flat sections. I am not sure why this is.

Okay, turns out I forgot to connect the ground probe of my scope for the latch output to ground, so it was reading static noise from the air surrounding it. Now I get the expected waveform:

In [14]: `InlineImage('goodwave.png', width=12, height=12, caption= "Figure 8, correct`



Now to find the delay:

We can note that the circuit in the delay section has an RC component; so the delay should theoretically be the time constant. The capacitor holds the signal for a little bit and then lets it go afterwards.

The expected delay is just the time constant of the RC portion, which is $11K * 1nF = 11 \text{ microseconds}$. I will explain why this is later in this writeup.

I will now scope over the delay portion of my circuit.

TS from my scope, I see that my input and output on the inverter for the delay portion is perfectly out of phase like so:

```
In [16]: #error waveform
InlineImage('wierdout.png', width=12, height=12, caption= "Figure 9, incorre
```

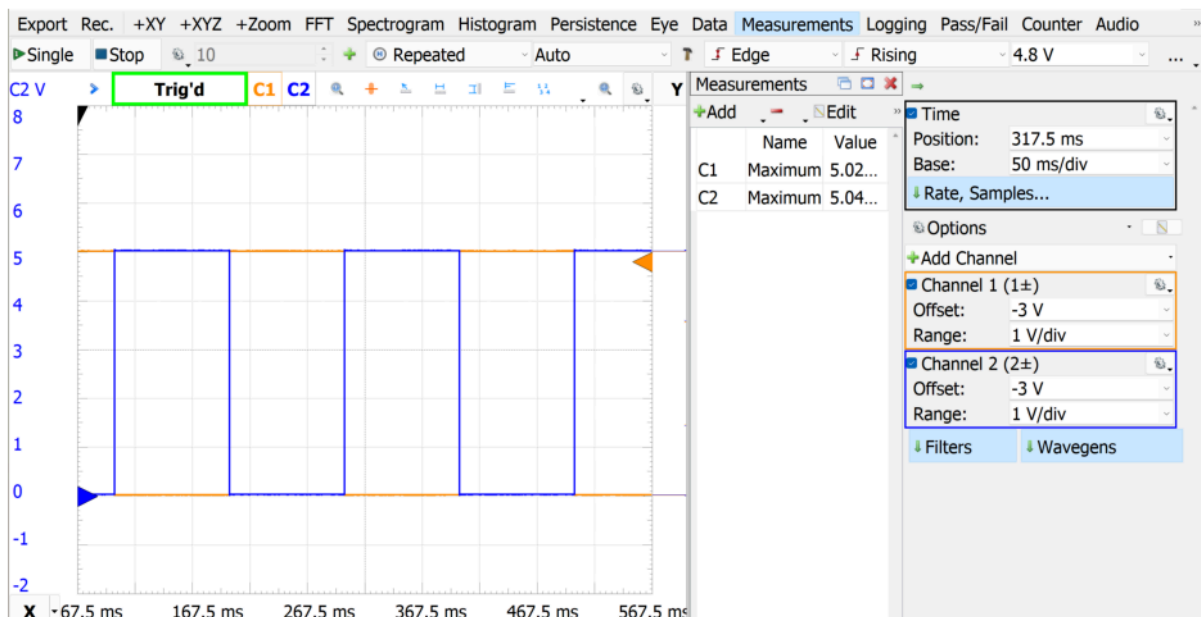


Figure 9, incorrect out of phase waveform scoping the delay input (orange) and delay output (blue).

First, I verified that my circuit is connected correctly. It turns out I had a capacitor and resistor in the reset portion of the circuit connected incorrectly. I fixed that, but there was no effect. This makes sense, as I am only scoping over the delay portion; the rest of the circuit afterwards should have no effect. However this saves me some troubleshooting later.

Next, I verified that my capacitors have the right value; maybe this affects the waveform? no effect.

After some thinking, I realize that this is basically just a regular inverting signal. Technically makes sense, but I should still be seeing a delay.

I asked for help, and it turns out my probes were connected wrong, and also my scope was a bit wonky and not zoomed in enough. Thanks Michael :)

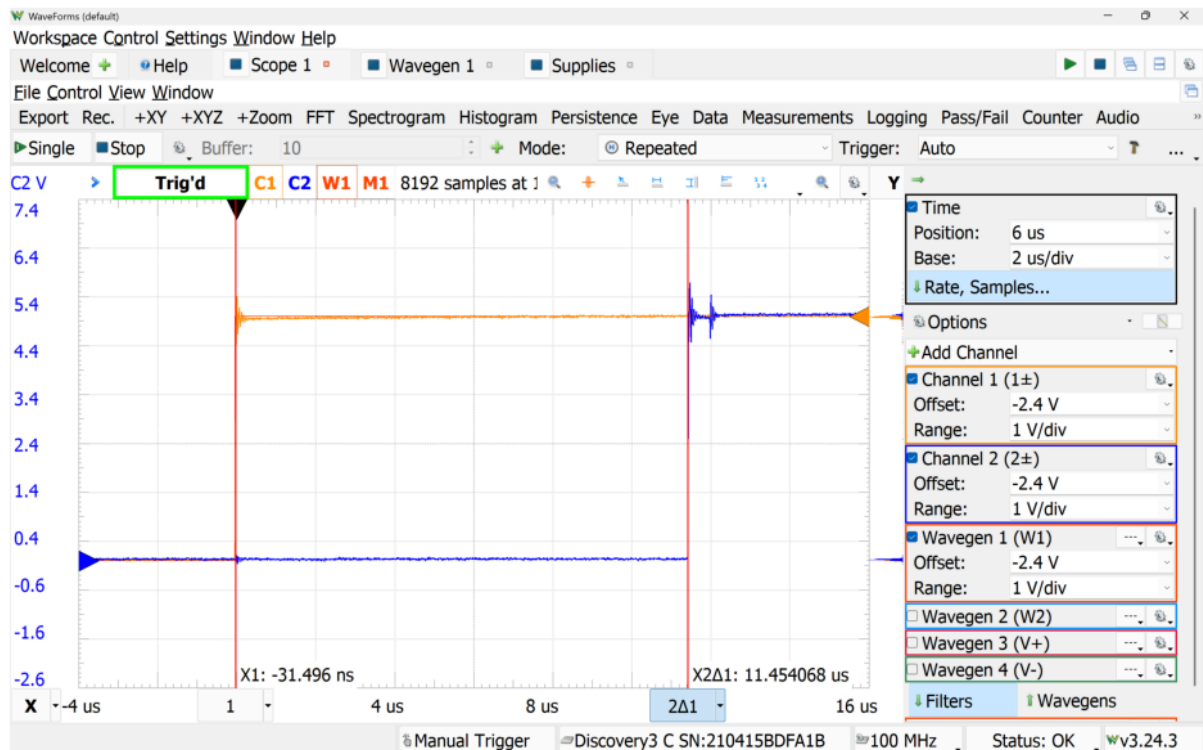
Now that I see the expected shape, I can use the cursors to find the actual delay in the circuit:

I will do this by zooming in, and placing the cursors to measure the time between when the input and output go to high/low.

TS Okay, turns out my delay is a bit short, at a value of 100ns. this is very short compared to the expected 11us. I asked for help again and we found that my resistor was not connected properly.

I will correct this. Here is what I read with the cursors with the resistor properly connected:

In [29]: `InlineImage('delayworks.png', width=12, height=12, caption= "Figure 10, Corr`



As we can see with my cursors, the delay between the signal of the input and the output is approximately 11.45us, which is very close to my theoretical value of 11 us.

How is a delay introduced?

I think that the delay is introduced by the resistor/capacitor prior to the inverter in the delay section of the circuit. My suspicion is that the capacitor charges, so

no input signal gets past the capacitor until it charges to a certain point. Once the capacitor charges to a certain point, the input signal can finally get through and reach the inverter of the delay circuit.

I would assume that this point is that threshold values we found in the previous lab, which will be different for rising and falling edge of the input. This is the hysteresis effect we observed prior.

Once the input signal switches to 0, the capacitor still holds charge, as it never has a chance to discharge while the input is still providing a current. Right after the input switches to 0, the capacitor is able to start discharging, which essentially maintains the signal for an extra amount of time. Once the threshold voltage is hit again by the discharge curve, the output signal finally disappears.

Theoretically, this extra amount of time for the discharge curve should be the same amount of time as it took for the capacitor to charge, which is the same as the delay we see in the input and is equal to the **time constant** of the resistor and capacitor network.

Why does this take the value of the time constant? Well, we know that the time constant is when the capacitor charges/discharges by roughly 63%. And given that we are using a 5V input wavegen, we know that while the capacitor is charging, $V_{in}(\tau) = \sim 3.15V$. while the capacitor is discharging, $V_{out}(\tau) = \sim 1.85V$.

This happens to be essentially exactly the values at which the inverter switches its output. SO: as long as we have a 5V input waveform, we can use any setup of R and C, and still have the delay be equal to the value of the time constant. We can alter the values of R and C to tweak this delay to what values we want!! (provided the time constant is not longer than the period of the input wave, at which point why bother).

IMPORTANT: we must also note that since we are using an inverter, when the input switches to high, the output switches to low. So for the RC curve, we will expect to see that the output signal is low after the capacitor charges to a certain point.

Anyways, if I find a falling edge to the signal, I should measure the same periodicity for the delay. And if I measure over the resistor/capacitor only, I should be able to see the actual RC charge or discharge curve.

```
In [30]: InlineImage('falling.png', width=12, height=12, caption= "Figure 11, falling
```




Figure 11, falling edge of delay signal, with the same delay value as the rising edge.

In [31]: `InlineImage('rccurve.png', width=12, height=12, caption= "Figure 12, RC curve`

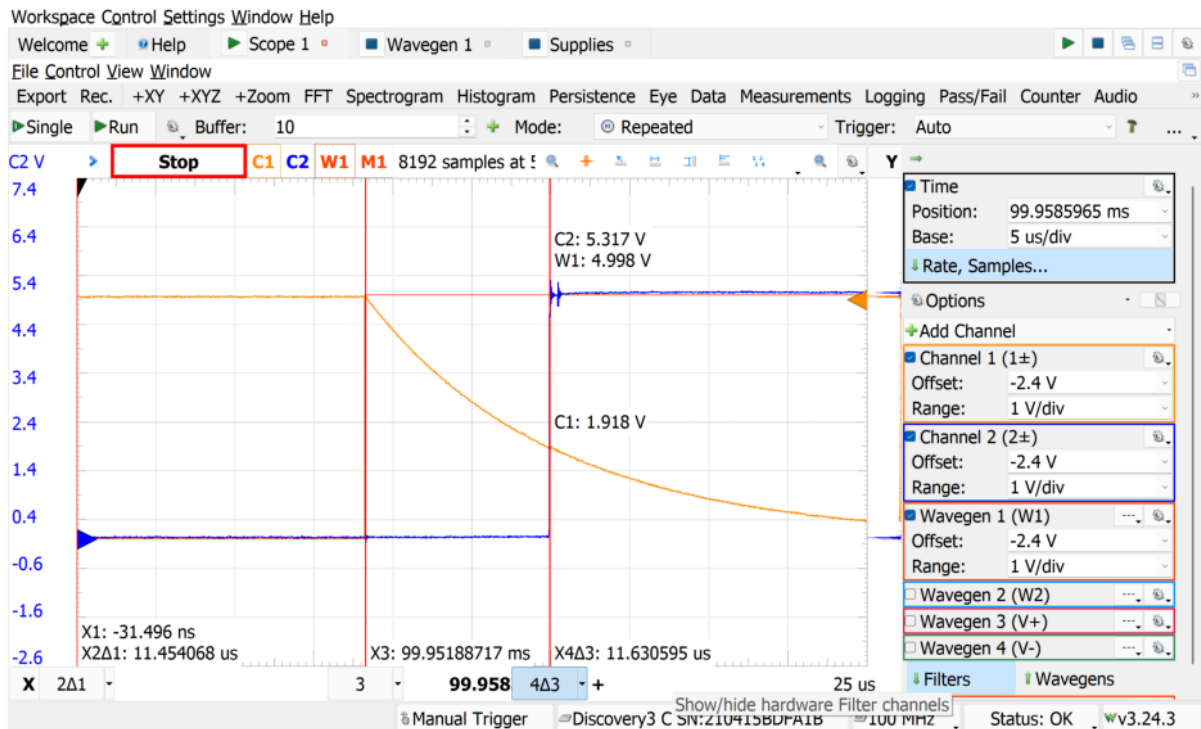


Figure 12, RC curve (orange), input wavegen (red) and output waveform (blue) of the delay circuit.

In [32]: `InlineImage('risingRC.png', width=12, height=12, caption= "Figure 13, rising`

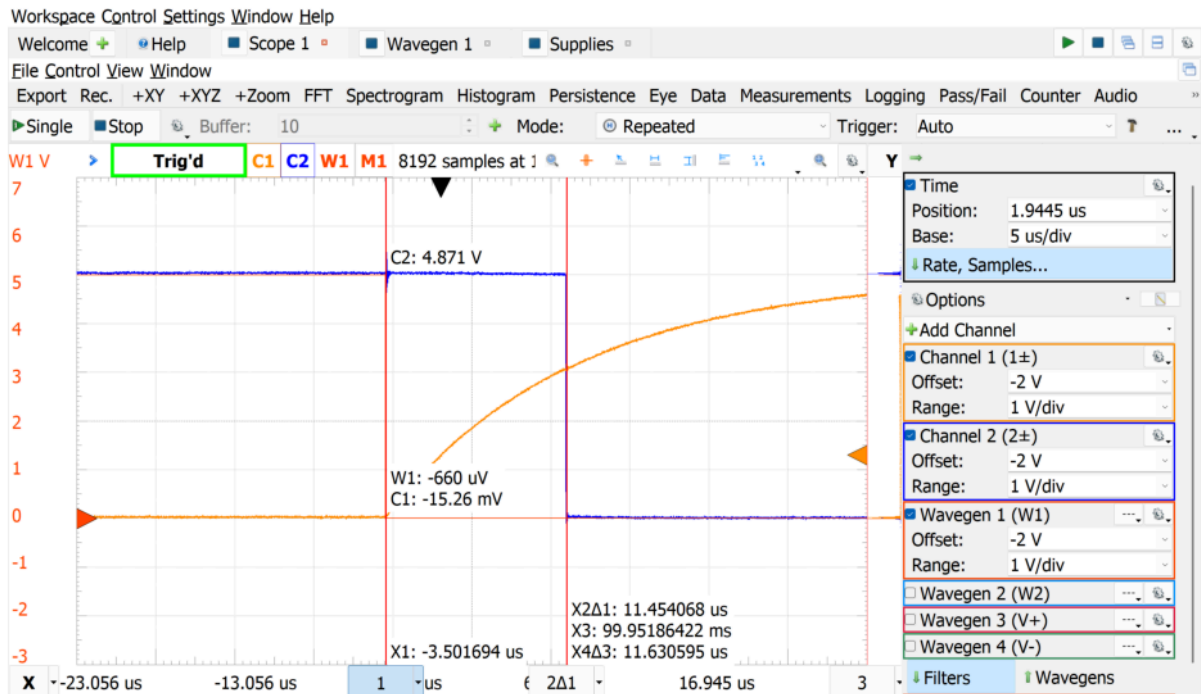


Figure 13, rising RC (orange), input wavegen (red), and output waveform (blue) of the delay circuit.

This is a visual verification of the time constant of the circuit, as well as the delay behaviour as a result of RC transient behaviour. We can also see that the delay looks approximately like where 1 time constant would be in the RC discharge curve.

Now onto the reset portion of the circuit:

For the schematic and photo, see above! I have grouped this part of the lab into one diagram and photo. This is since it all involves the same IC chip.

Here is a visual of the reset pulse.

```
In [33]: #delay pulse waveform.
InlineImage('pulsed.png', width=12, height=12, caption= "Figure 14, reset pu
```




Figure 14, reset pulse output(green), reset pulse input (blue), and input square wave (red).

How does the reset circuit work?

As we can see from the screenshots above, we have a small pulse of around 550 ns. This is a small pulse that is sent out, but why does it happen?

Well, we can examine the circuit and see that there is a similar RC component before the inverter of the reset portion. However, the order is simply switched. we can still find the time constant, which is around 517 ns. ($47 \text{ pF} * 11 \text{ k}$). Basically, it follows a similar principle to the delay portion of the circuit. At a given edge of the input wave (rising edges), the rc portion of the circuit outputs a small very short spike which is approximately the duration of the time constant. This is because the capacitor initially blocks a constant signal, and only changes (discharges) when an edge is detected (change in voltage) in the circuit. The capacitor discharges quickly and waits for the next edge. So it doesn't reset on a falling edge; otherwise it would send 2 reset pulses in one reset pulse period (for input waveform).

Why are there 2 inverters?

The first one is likely to clean the signal up and apply hysteresis, which adds a layer of noise immunity. However, this inverts the signal, and I am guessing that is why we have a second inverter. We want the signal to be high for a reset pulse, just for ease of dealing with and clearer logic. Essentially we guarantee the output logic truth is the same as the input logic truth, all while cleaning up/reinforcing our signal.

post-lab tony here:

what does this actually do in the context of the circuit?

Basically, we have a circuit that needs to save the revolutions of the motor in a DLatch. However, this is fed by a counter which we want to reset after certain intervals of time. In order to give the time for the data to be latched or saved, we need this delay generator in tandem for the reset pulse; so that the reset pulse is sent only after the data has been latched, and not before; otherwise we would not be able to latch the revolutions and as such determine the speed at which the motor is spinning in order to create the rpm modulating feedback loop.

Now, I will move onto the next portion of the circuit:

##2.2 Task 2, Counter, Dlatch, and DAC

For this task: I will use

- a 74HCT393 divBy16 counter
- a 74HCT273 latch
- a 4610X-R2R-103LF 8 bit DAC
- a UA741CP OpAmp

Once again, I am including this photo of the circuit layout from the lab manual for easy reference. I will include the circuit diagram and photo for each IC separately for clarity.

In [34]: `InlineImage('fullcircuit.png', width=12, height=12, caption= "Figure 15, Cou`

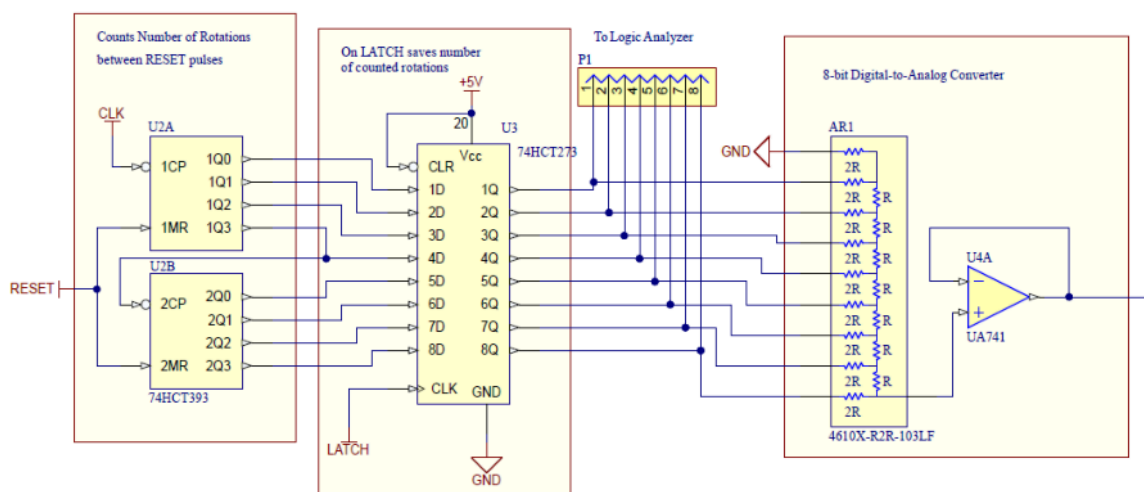


Figure 15, Counter, Latch, and DAC circuit diagram from lab manual.

Here is a full circuit photo as well as diagram for this section of the lab. I have built the counter portions as well as the R2R in one go to save time:

NOTE: for the DLatch, I have kept the circuit diagram as a pinout diagram of the chip, rather than drawing out the full logic flow/internal connections. Here is a basic rundown of how it works; its composed of a bunch of NAND gates with strategic 'backflow' which causes a certain value to be 'latched'. Recall previous lab, where we used a DLatch that can store either HIGH or LOW. Here, we have a D latch that can store values up to a 16 bit equivalence; essentially we have 16 D latches, one for each 'bit' responsible for counting the revolution of the motor. This is important as we want to be able to store the full range of values our counter can count up to.

```
In [46]: InlineImage('latchcount.png', width=12, height=12, caption= "Figure 16, phot
```

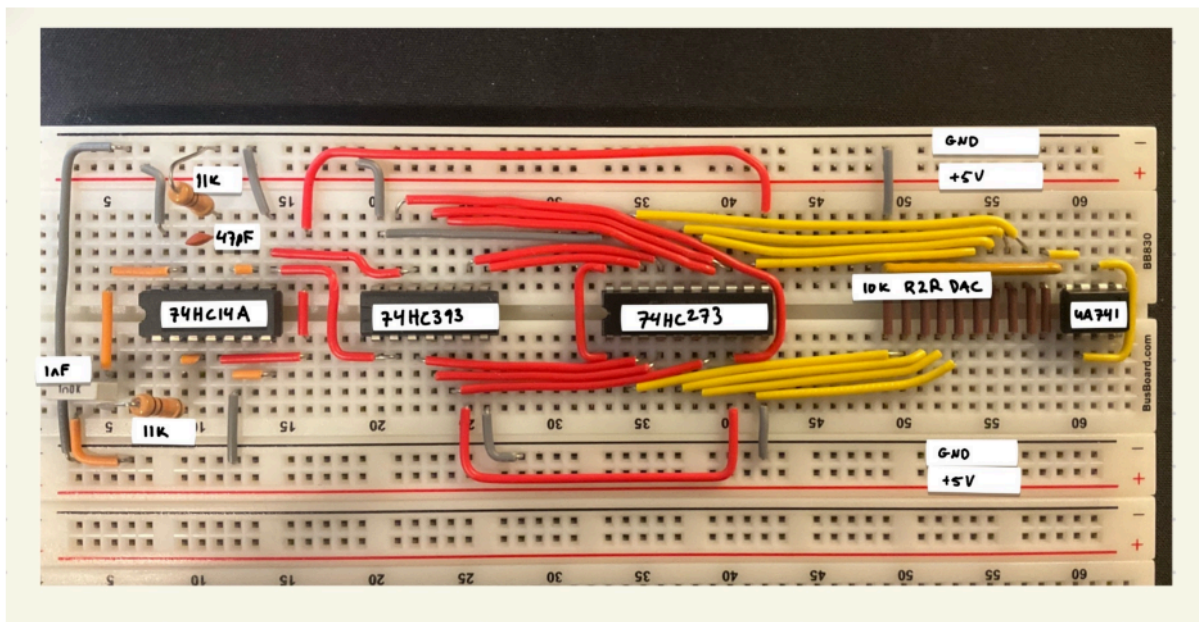
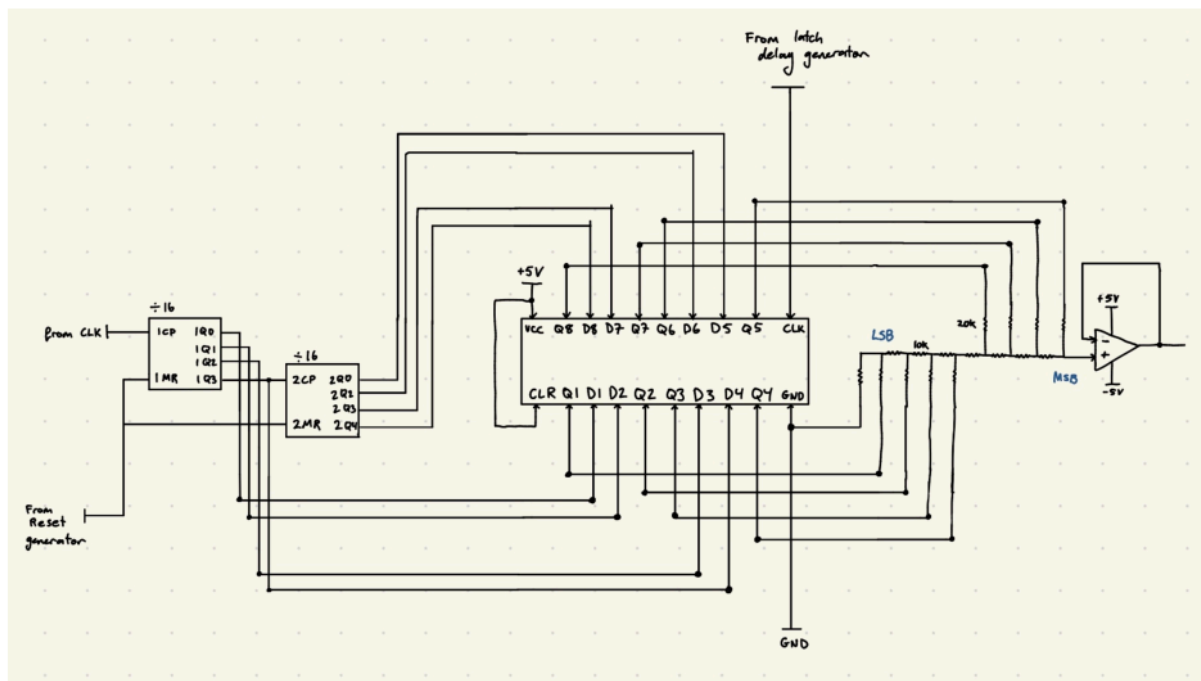


Figure 16, photo of my full circuit for this section of the lab. Includes counter, DLatch, r2r dac, and op amp.

NOTE: the connections and chip on the very left are insignificant for this specific section of the lab; those are the delay and reset pulse generators from the previous sections.

```
In [44]: InlineImage('fulldiagram.png', width=12, height=12, caption= "Figure 17, cir
```



Here are the pinout diagrams for the parts I will use in this section, from their datasheets:

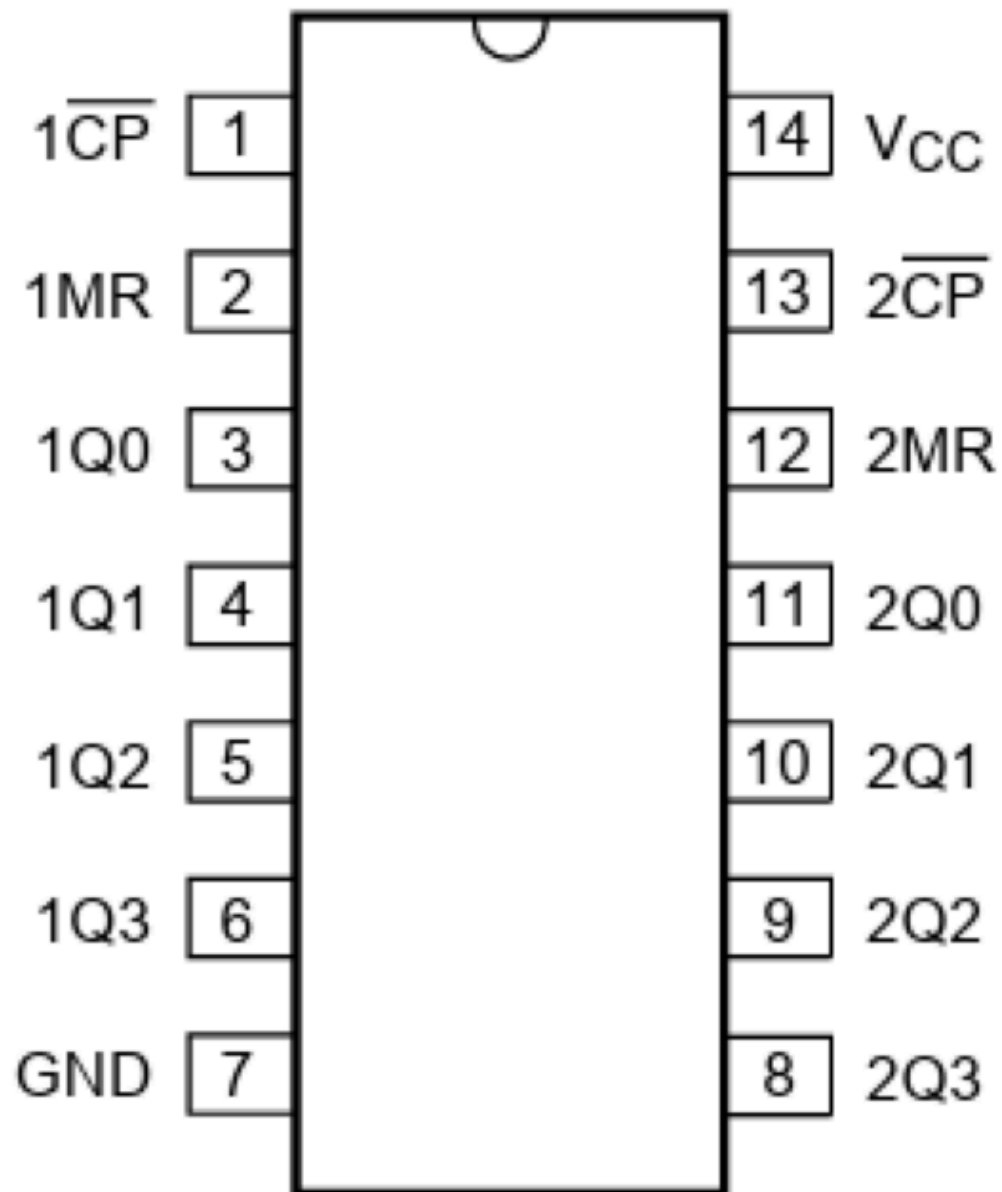
```
In [48]: InlineImage('counterpin.png', width=12, height=12, caption= "Figure 18, 74HC  

InlineImage('ripple.png', width=12, height=12, caption= "Figure 19, 74HCT273  

InlineImage('dac.png', width=12, height=12, caption= "Figure 20, 4610X-R2R-1  

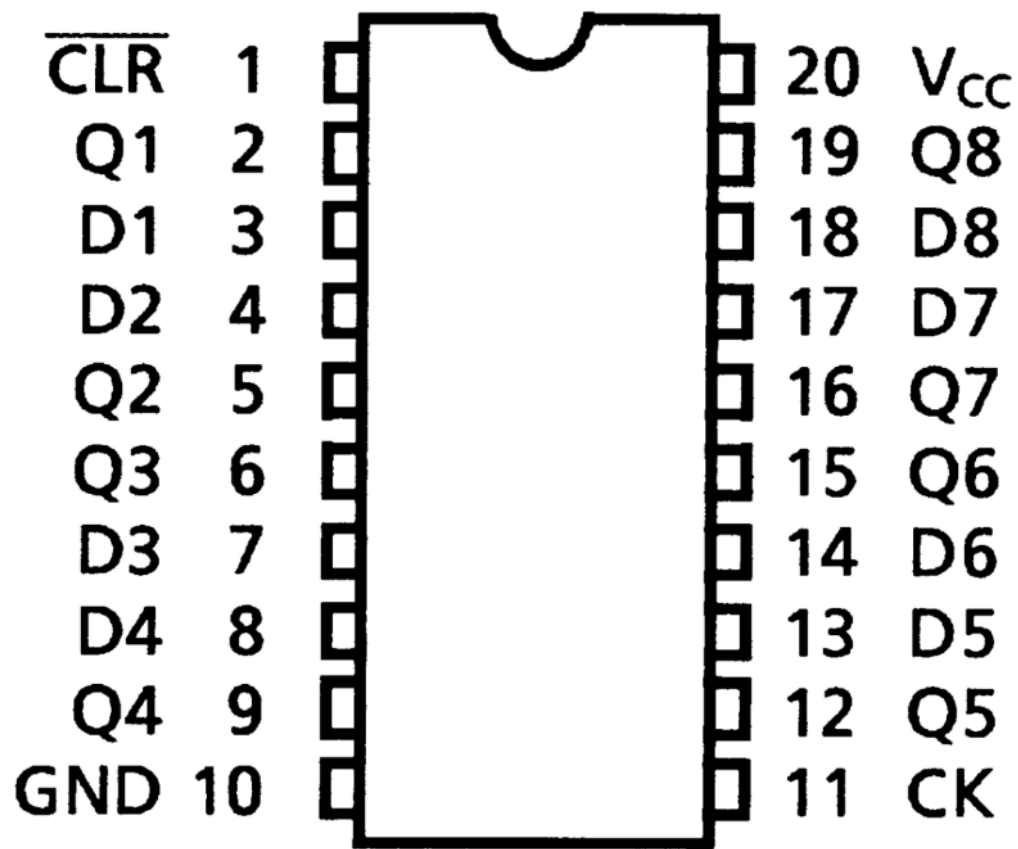
InlineImage('opamp.png', width=12, height=12, caption= "Figure 21, UA741CP C
```

74HC393
74HCT393



aaa-007518

Figure 18, 74HC393 DivBy16 counter pinout diagram.



(TOP VIEW)

Figure 19, 74HCT273 Latch pinout diagram.

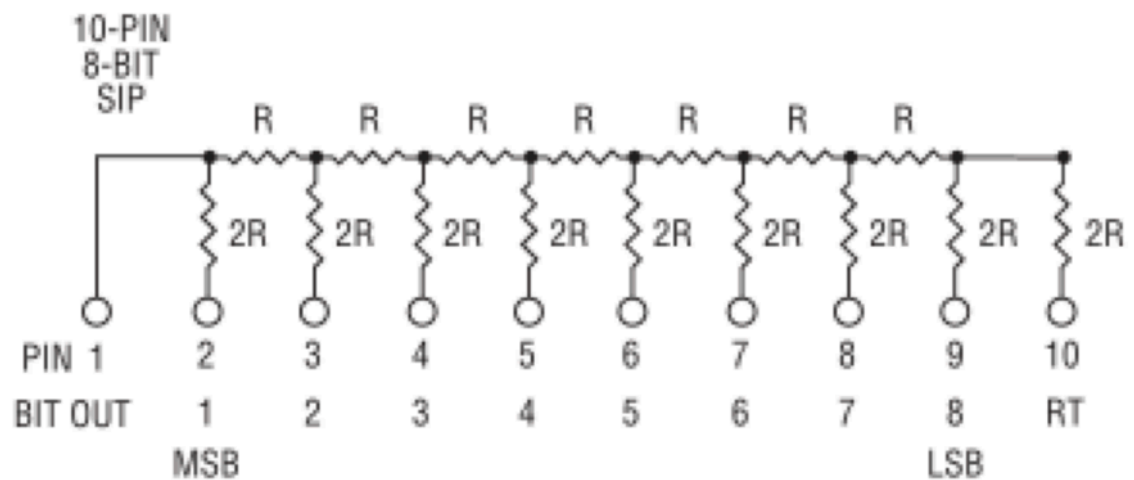


Figure 20, 4610X-R2R-103LF 8 bit DAC pinout diagram.

**uA741C D, P, or PS Package
8-Pin SOIC, PDIP, SO
Top View**

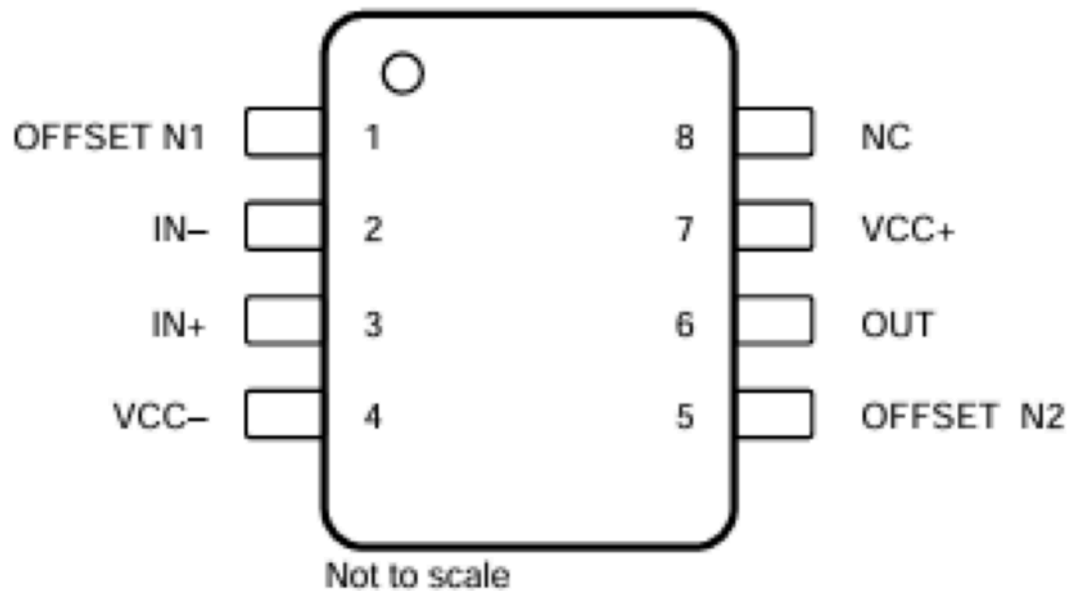


Figure 21, UA741CP OpAmp pinout diagram.

Now that I have built my circuit, I will analyze it using the logic and patterns apps in waveforms.

I will start with the counter.

Here is a photo of my counter. For the diagram, see the start of this section in the writeup. I have only highlighted relevant parts for clarity:

In [38]: `InlineImage('counter2.png', width=12, height=12, caption= "Figure 22, counte`

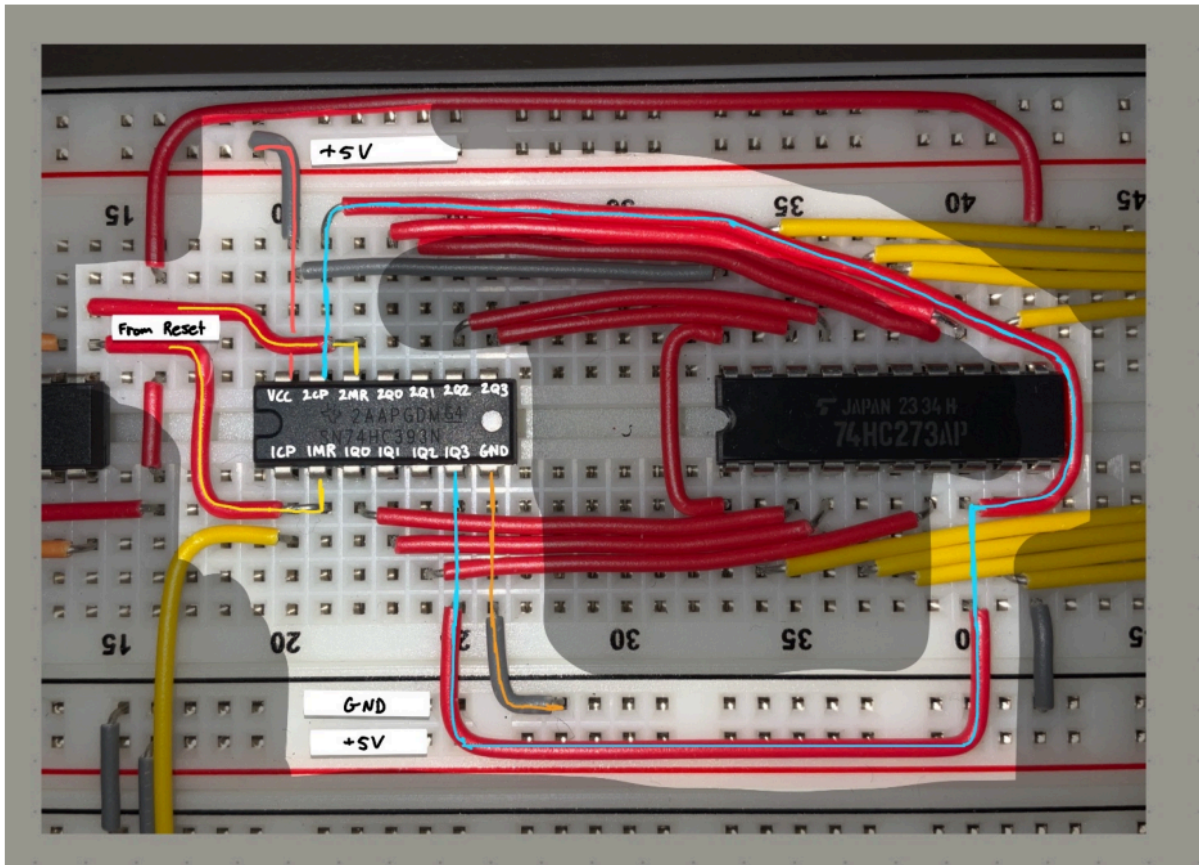


Figure 22, counter circuit photo.

First, I will verify the functionality of the counter by scoping its outputs directly:

In [42]: `InlineImage('counterwork.png', width=12, height=12, caption= "Figure 23, counter functionality")`

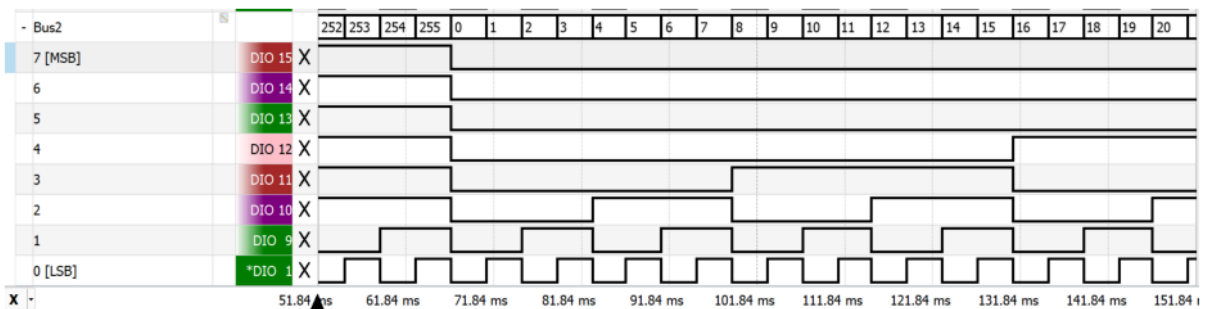


Figure 23, counter functionality.

What is the max value the counter can count to?

Theoretically, as it is an 8 bit counter, it should be able to count up to 255. after which, it will reset to 0 on the 256th count (2 to the power of 8). with 8 bits, we can represent $2^8 = 256$ different values, so 0 to 255 inclusive. After hitting the maximum value of 255, it resets to 0. and continues counting once again.

Now that I have the counter working, I will begin working on my latch.

Here is a photo of the latch. Again, see start of section for circuit diagram.


```
In [49]: #latch photo here
InlineImage('latchphoto00.png', width=12, height=12, caption= "Figure 24, photo of latch circuit showing connections from counter.
```

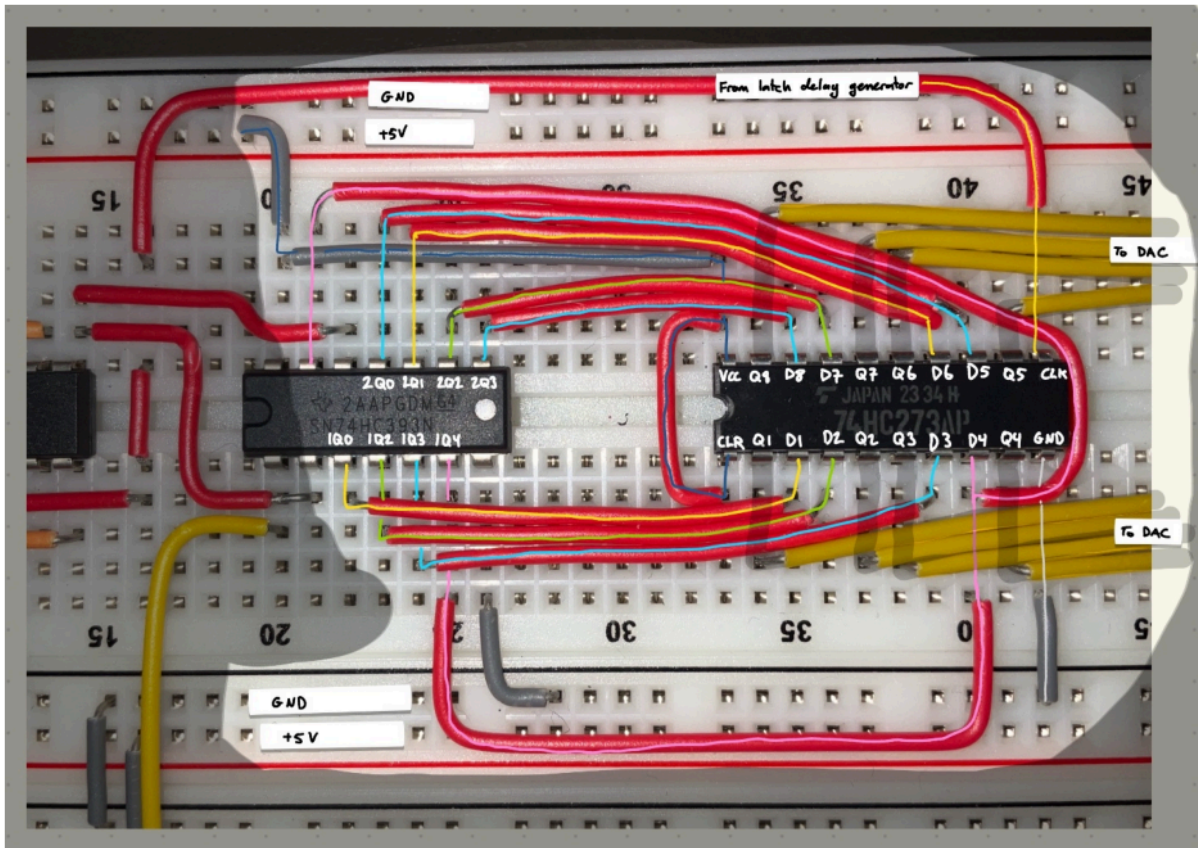


Figure 24, photo of latch circuit showing connections from counter.

I connected 8 DIO pins to the circuit as specified in the lab manual (see image below). Then, I drove the circuit from the very beginning, using the 5Hz square wave input. I have a constant 0 wave DIO pin connected to RESET to turn its functionality off for now. I am then driving the lsb on the counter with the DIO with a 250Hz clock input.

Now that everything is connected, I will use the logic app to analyze its function.

```
In [50]: InlineImage('diopins.png', width=12, height=12, caption= "Figure 25, DIO pin connections to the counter.
```

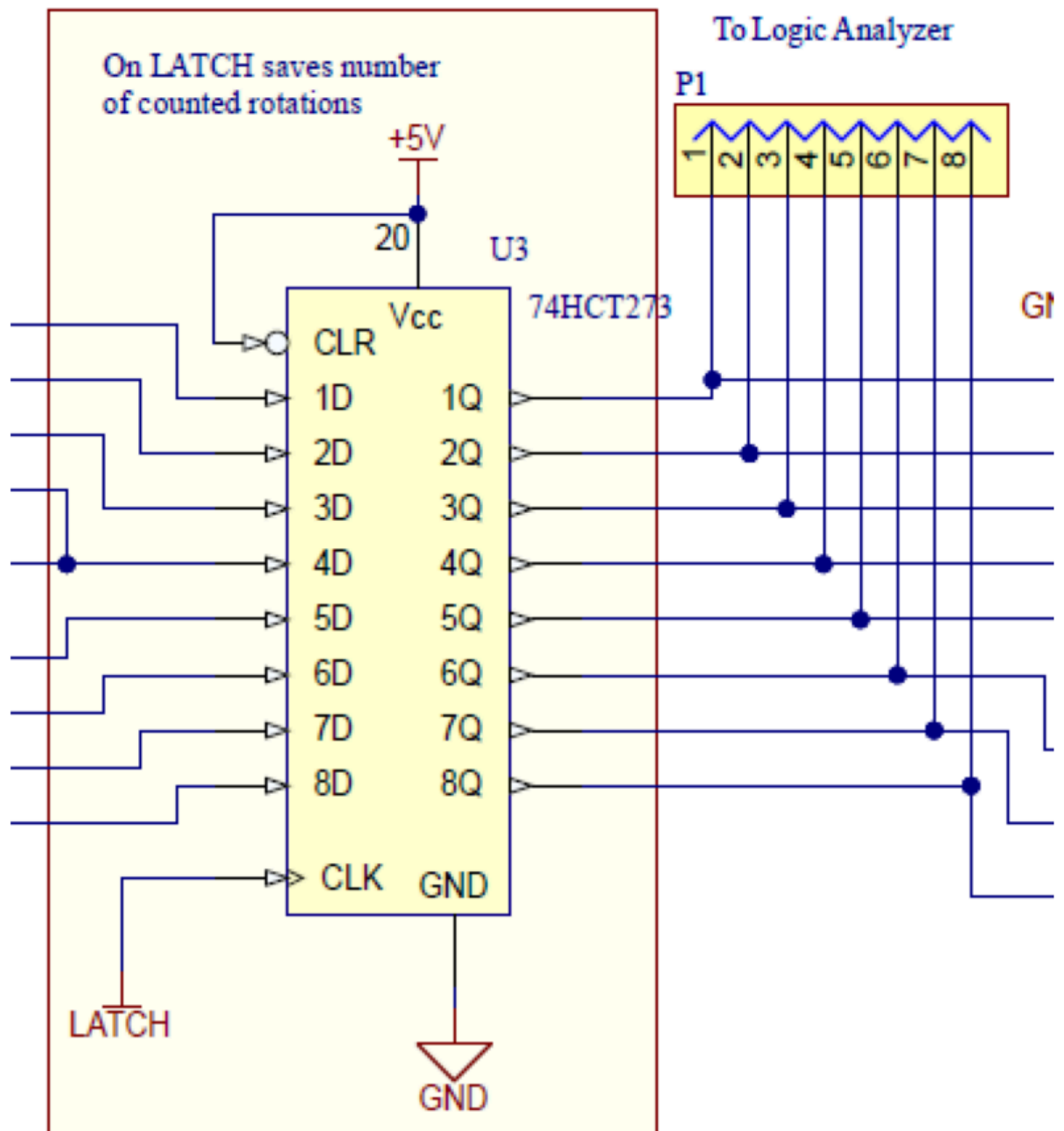


Figure 25, DIO pin setup after the latch.

TS

I am not seeing an expected output for the latch. I have connected everything according to the given circuit diagram in the lab manual, but I am only seeing the lsb toggle states. Furthermore, the BCD BUS only reads toggling between 34 and 35, analogous to when the lsb is 0 vs 1. This is not expected.

I would expect that the latch saves 1 constant value, rather than acting as a counter somehow. However, I know that my reset pulse works, and that my

waveform driving CLK is not changing frequencies magically. SO there is likely something wrong with how I have wired things.

Here is what I try in order:

- The first step was verifying circuit connections. I traced each path with more care, and found that there is theoretically nothing wrong with the way the circuit is connected. From this, I will conclude that it must be an issue with how I am driving the circuit, or another issue unrelated to circuit connections.
- The next step is verifying that all of the power sources and grounds are connected properly. Everything is connected properly. My power source is 5V and I am not getting any red dots in the waveforms app showing input lower than that. so nothing is short circuited. My waveform is on, and is a 5Hz square wave from 0 to 5V. This part is the same as the first section in this lab and is unchanged. So from these, I can say that the power source/ground connections are correct.

Now, I have a few main suspicions:

- I will note that I have not re-analyzed my connections along the R2R resistor ladder/DAC. I will do that as well, as that could be part of the issue.
 - I will also note that I could be driving the clock wrong, and maybe my DIO pins are the incorrectly connected portions.
1. checking the DIO/R2R connections: How I understand it, the LSB is beside Ground. The MSB is beside the output, which connects to the UA741 OpAmp at the end of the circuit. 1Q on the counter is the LSB, and 8Q is the MSB. Okay, these are connected properly.
 2. checking if I am driving the circuit properly with the DIO pins: Ok, so I have made sure each DIO pin is connected. I re-read the lab manual, and found that I was driving the circuit incorrectly. I had DIO 1 run a 250Hz clock, but this pin was connected directly to 1Q and the LSB on the R2R DAC. I think I was supposed to drive the circuit from the pin labelled CLK in the prelab diagram. I will try this.

TS

Upon doing the above changes, I notice few changes. I am driving from CLK now (seems quite obvious now and I am not quite sure what I was doing before). However, the Bus reads a constant value of 32. Additionally, I notice that DIO 1 and DIO 4 and DIO 5 are all at 1, while the other DIO are at 0. (DIO 0 is my LSB, DIO 7 is my MSB). This does not add up. theoretically, these bits toggled should

be $4 + 36 + 64 = 100$, but the bus reads 32. So something is wrong with the DIO connection I think. Maybe I missed something the first time.

Now my bus reading fluctuates between 128 and 130. bit 3, 6, and 7 are on constantly now, while bit 1 fluctuates. Okay, so I have the CLK input in my Bus. That should not happen.

Here, I switched some pins around. DIO 0 is my CLK now, and DIO 1-8 are my 8 DIO connections for the R2R DAC. Upon removing the DIO 0 (CLK) from my Bus: I am back to seeing a constant value of 32, while DIO 2, 5, 6 are on. There is a clear mismatch, but now at least I know that it is not because of my bus settings in the app. So I must have something connected incorrectly in the circuit itself, regarding the DIO pins.

I am now verifying that my pin mapping is correct. I will do this by testing each pin value separately by itself as having either 0 or 1, and then verifying that the bus reading is correct.

Ok, so everything up to DIO 4 is correct. DIO 5 however, gives me a reading of 20 rather than 32 as I would expect. I double checked connections again just to be safe, looks like everything is as it should be.

I asked a friend, it turns out my issues are from using the BCD bus type rather than the decimal bus type. Now, pin mapping is correct and I get the expected bus value.

Running the counter again, I get that bits 2, 5 and 6 are constantly on and I still have a constant bus reading of 100. I will double check connections for these bits and their counter connections.

I grounded the reset, rather than having a dio pin force it to 0. Now, I get that the counter counts in regular intervals. However, it counts by 100 each time and starts at a different value after each reset.

```
In [51]: InlineImage('upby100.png', width=12, height=12, caption= "Figure 26, counter
```

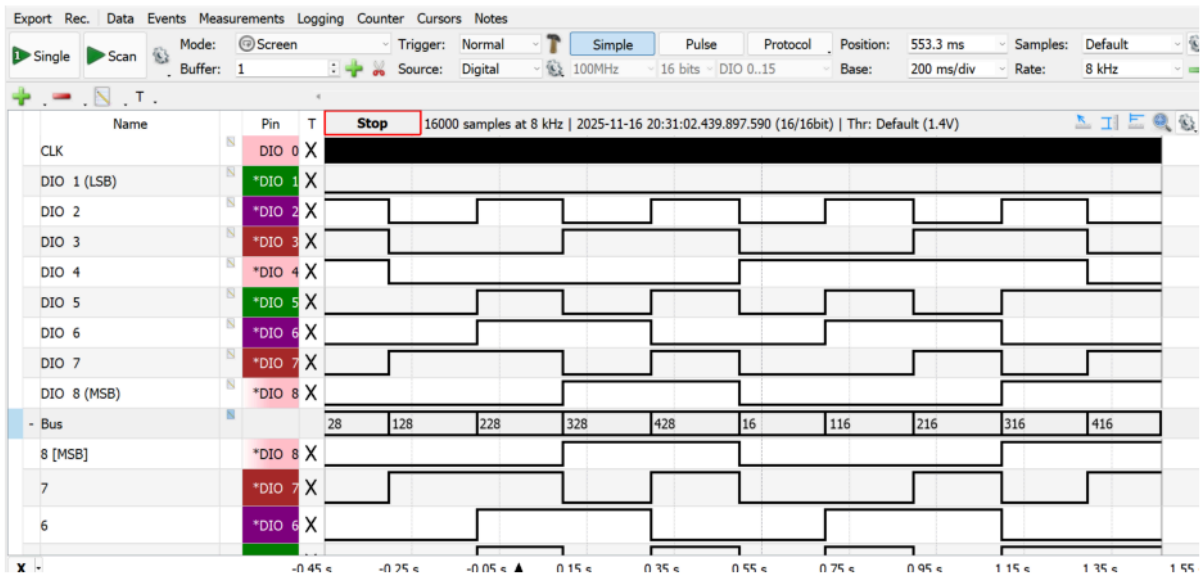


Figure 26, counter working, but counting incorrectly..

I will double check the counter itself.

I checked the counter, it functions as I expect; counts from 0 to 255 and rolls over and then starts counting again. I have now eliminated a lot of possibilities as to what the issue can be. I am now suspecting that it is the the 74HC273 or the connection to the latch part of the circuit. Here is a screenshot of my logic flow for the 8 bit counter:

In [52]: `InlineImage('counterwork.png', width=12, height=12, caption= "Figure 27, isc`

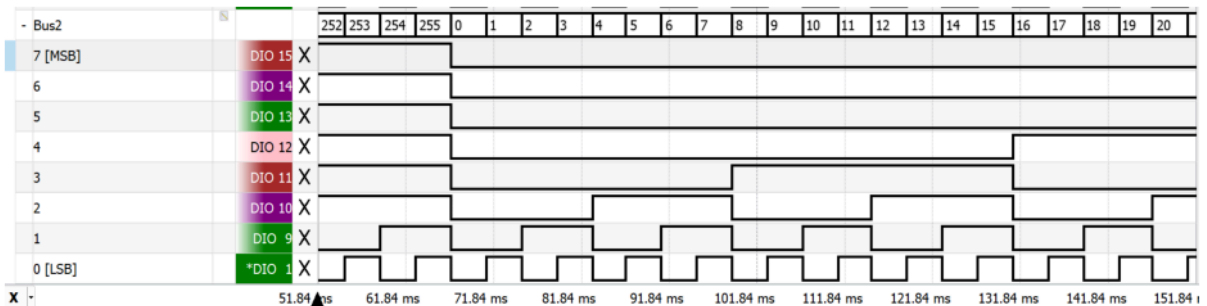


Figure 27, isolated counter counting to 255 and rolling over to 0. correct behaviour.

This is the next day, I am continuing to troubleshoot the latch with a fresh start.

I have now encountered an issue with my counter itself.

Here is how I have laid my circuit out:

- I have an input square wave of 5Hz going into the delay and reset generator at the very beginning of the circuit.
- I have an input square wave of 5Hz going into the CLK of the counter. This will emulate the motor input (revolutions).

- I am using a DIO pin at each output of the counter, but I see wierd behaviour upon adding the pins to a BUS to read the decimal count value.

I have the following issues:

- my counter counts up by 2
- it seems to count 62, then jumps to 128 and continues counting up by 2.
- after jumping to 128, it counts up to over 400, which is very unexpected... I am very confused as to why this may happen, and here is my troubleshooting for it below:

First, I try setting up my DIO pins over the counter again. I have reset grounded for now, just to verify that my counter is connected/rolling over properly:

- I am getting that the counter counts up by 2, up to a large number in the 500s (I am guessing 512). I am unable to observe this with how slow my logic app runs on my computer.
- It resets to a number that I am also unable to observe given the limited range of values, and the absurdly slow refresh rate of the logic app. I can only guess it is either 0 or 2, from how it has been behaving previously.
- after a while, I chanced on confirming this behaviour; it counts by 2 to 510, rolls over to 0. This is analogous to counting to 255 by 1. So I think that I just have my bus set up incorrectly in the app, but otherwise my counter is counting the right number of times. I am just reading it wrong:

In [53]: `InlineImage('count510.png', width=12, height=12, caption= "Figure 28, counti`

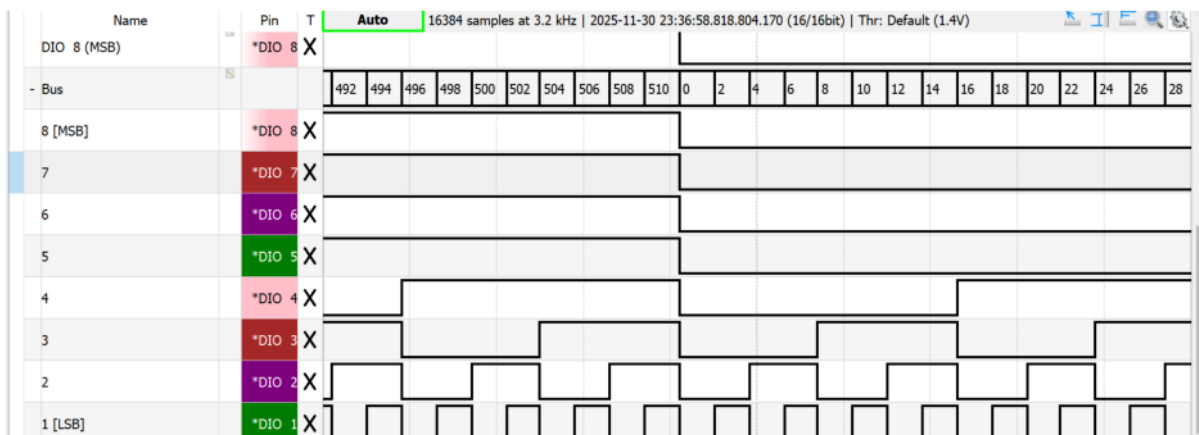


Figure 28, counting from 0 to 510 by 2.

Basically I am one bit off for my bus. I double checked connections for the billionth time, and they are correct. I GUARANTEE THE ISSUE IS IN MY BUS SETUP NOW!!!!

Ok I found it. This is really dumb but basically in my bus, there is an MSB and LSB toggle option at the bottom. I had LSB set to 1, and MSB set to 8. This shifted the whole counter 1 bit larger. I have set LSB to 0, and MSB to 7. Now it works. Below is my setting for it so I know for the future:

In [54]: `InlineImage('bussetting.png', width=12, height=12, caption= "Figure 29, BUS`

Figure 29, BUS settings.

Not sure how this changed, but I found that I can change it with my scroll wheel. I suspect I accidentally scrolled while moving my mouse over those boxes at some point in time.

Now that I have verified my counter works, I will move back to checking latch functionality.

In [55]: `InlineImage('again.png', width=12, height=12, caption= "Figure 30, counter w`



Figure 30, counter working again.

For this step, I am moving my pins back to where they are in the lab manual (see above figure 16). Here, I want to verify a few things:

- my reset pulse works
- I get a proper value being latched (I expect this to correspond with the ratio between the frequency of my reset pulse and the AWG connected to the count input)
- The output waveform at the very end is a constant value that corresponds with the latch value; it should not be a sawtooth/ladder/increasing waveform.

bruh.

I forgot to change my input wavegens to operate from 0 to 5v. I spent so many hours and this was the root cause. So nothing was wrong with my wiring, I just wasn't providing a signal that the chips could read.

```
In [56]: InlineImage('newset.png', width=12, height=12, caption= "Figure 31, input wa
```

Channel 1 (W1)

 Run ☒ Enable Simple ▼ Idle:

Type: □ Square ▼ 

Frequency: 5 Hz ▼

Period: 200 ms ▼

Amplitude: 2.5 V ▼

Offset: 2.5 V ▼

Symmetry: 50 % ▼

Phase: 0 ° ▼

Channel 2 (W2)

 Run ☒ Enable Simple ▼ Idle:

Type: □ Square ▼ 

Frequency: 55 Hz ▼

Period: 200 ms ▼

Amplitude: 2.5 V ▼

Offset: 2.5 V ▼

Symmetry: 50 % ▼

Phase: 0 ° ▼

Figure 31, input wavegen settings.

Ok, now I know my latch works. Basically, I have a wavegen input to the CLK input of my counter. I also have a wavegen input to my reset pulse. The value that should be latched is the the ratio of the frequencies between the wavegen to CLK and the wavegen of the reset counter. For example, if I run the CLK at 500Hz, and the reset at 5Hz, then I should expect to see a latched value of 100. (the 'motor' completes 500 rotations before the reset pulse triggers).

Now for the OpAmp portion of the circuit. As always, refer to the top of this section for the complete circuit diagram.

Anyways; here, I have an R-2R resistor ladder which converts the latch value (which is a digital value) into an analogue voltage value. I suppose I can think of this as a 'mapping' for digital to analogue. I would assume that it is a linear relationship, where 5V is divided into the 256 values the counter/latch can hold. The smallest step size of this analogue voltage is $5V/256$.

Note: my power supply is not exactly 5V as my computer can't handle it. it is closer to 4.5 ish volts.

This voltage is then given to the OpAmp, which kind of acts to make the signal stronger/clearer, before handing it off to the error amplifier.

Here is a photo of my OpAmp, along with the R-2R and the inputs to the R-2R from the DLatch.

```
In [68]: #R-2R and opAmp here.  
InlineImage('dacandamp.png', width=12, height=12, caption= "Figure 32, R-2R
```

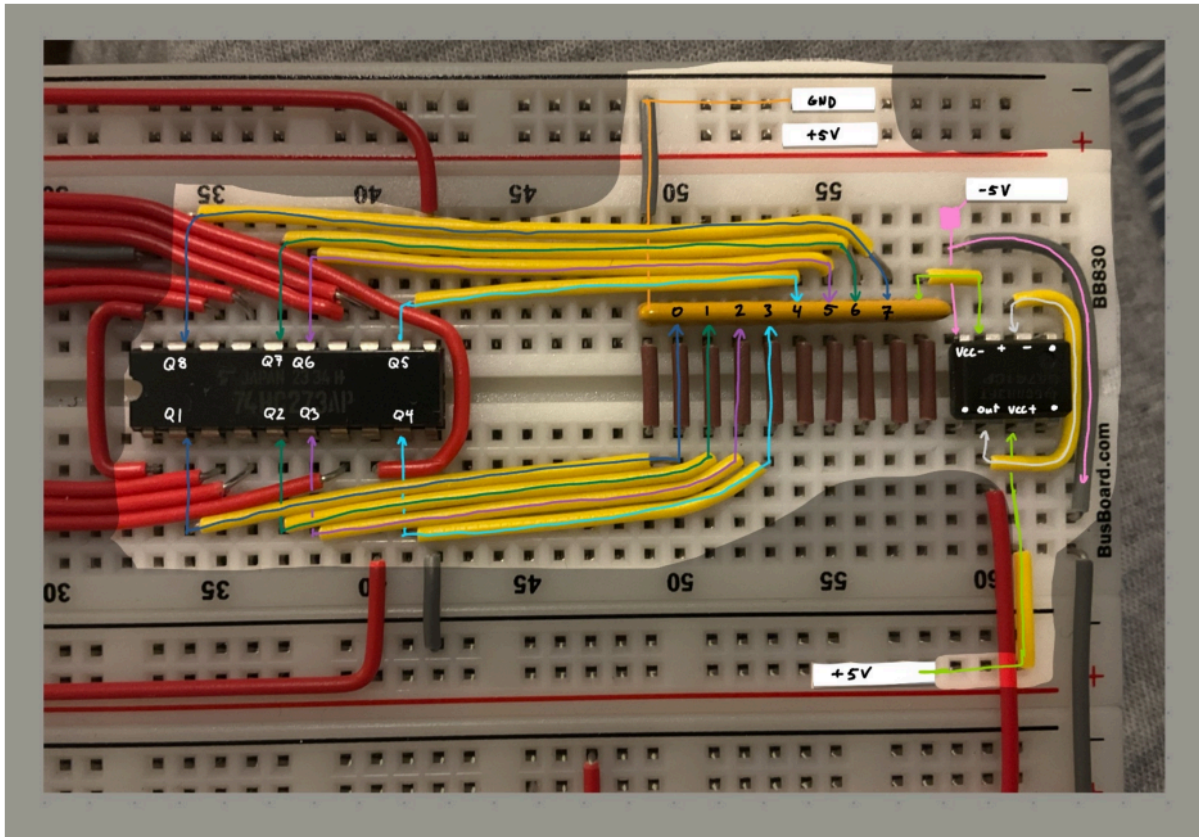


Figure 32, R-2R and OpAmp portion of the signal for conversion of digital input to voltage output. Connections from Latch are traced.

Now, I will start to work on the output of the OpAmp at the end of the R2R dac. If I were to scope the output of this OpAmp, I should expect to see a constant voltage that is proportional to the latched value. The higher the voltage, the more times the motor has rotated prior to the reset signal triggering.

I have set up a scope at the output of the OpAmp, here is what I see:

In [60]: `InlineImage('opout.png', width=12, height=12, caption= "Figure 33, opAmp out`

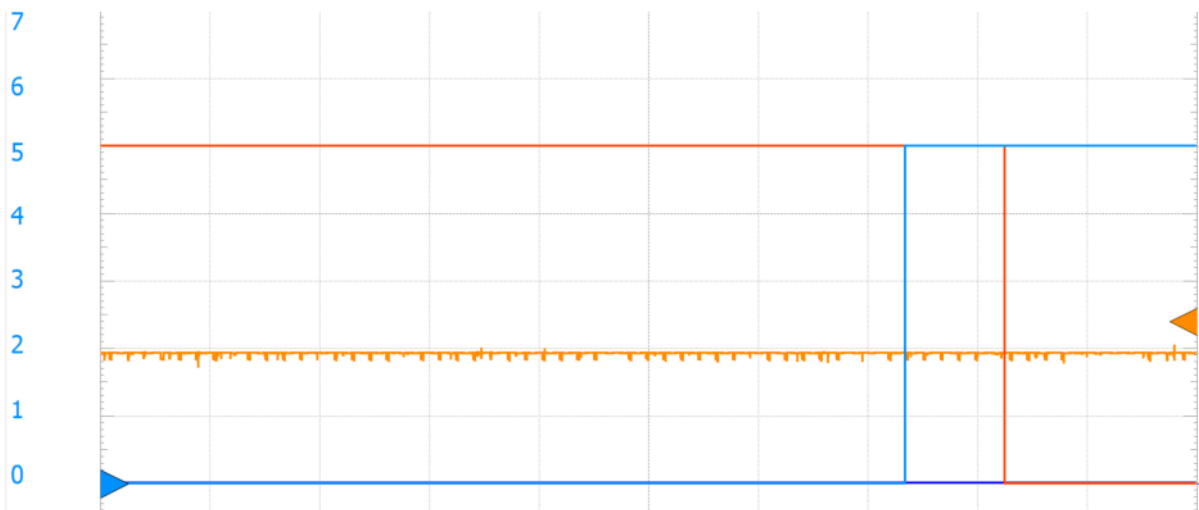


Figure 33, opAmp output (orange) with a 5Hz reset and 5Hz CLK input. ignore blue and orange (i forgot to turn them off for the screenshot).

As we can see, the output is a very noisy but generally constant value. However, given that my input waveforms are both 5Hz, I would expect to see a horizontal line very close to 0.

SO there is obviously something wrong with my circuit.

TS

I there are a few things that I am suspecting. Given that I have just verified functionality of the DLatch in conjunction with the reset pulse prior to this point, it can only be one of 2 things:

The circuit portion involving the R2R DAC, or the OpAmp itself.

first, I will check the connections of the R2R dac from the DLatch. I have a suspicion that maybe the latch is latching the right value, but maybe the data/bits of the digital signal are being incorrectly fed into the R2R which causes a conversion into an incorrect analogue value.

I have verified R2R DAC connections. I have confidence that now, the issue is with the OpAmp. I will double check the OpAmp connections first.

Upon doing so, I have found that my V- rail was connected to ground, rather than -5V. This would prevent the OpAmp from functioning properly; it basically sets the limit for that rail to be 0V rather than -5V, which can cause asymmetry and clipping. I assume both of those are why I was seeing what I was seeing: the offset of the waveform despite having latched a value of 1, and no change; the whole signal seemed to be getting clipped...?

I changed the OpAmp connection for VCC-, and now I get a waveform that changes with the CLK frequency and thus the latched value.

I am testing it with a 500Hz CLK, meaning I latch a value of 100. I should expect to see a value that somewhat corresponds to $100/256$ of 4.5V (approx my power supply from my laptop), which is approx 1.75V. It works, but there is still noise:

In [61]: `InlineImage('railsfixed.png', width=12, height=12, caption= "Figure 34, OpAmp")`

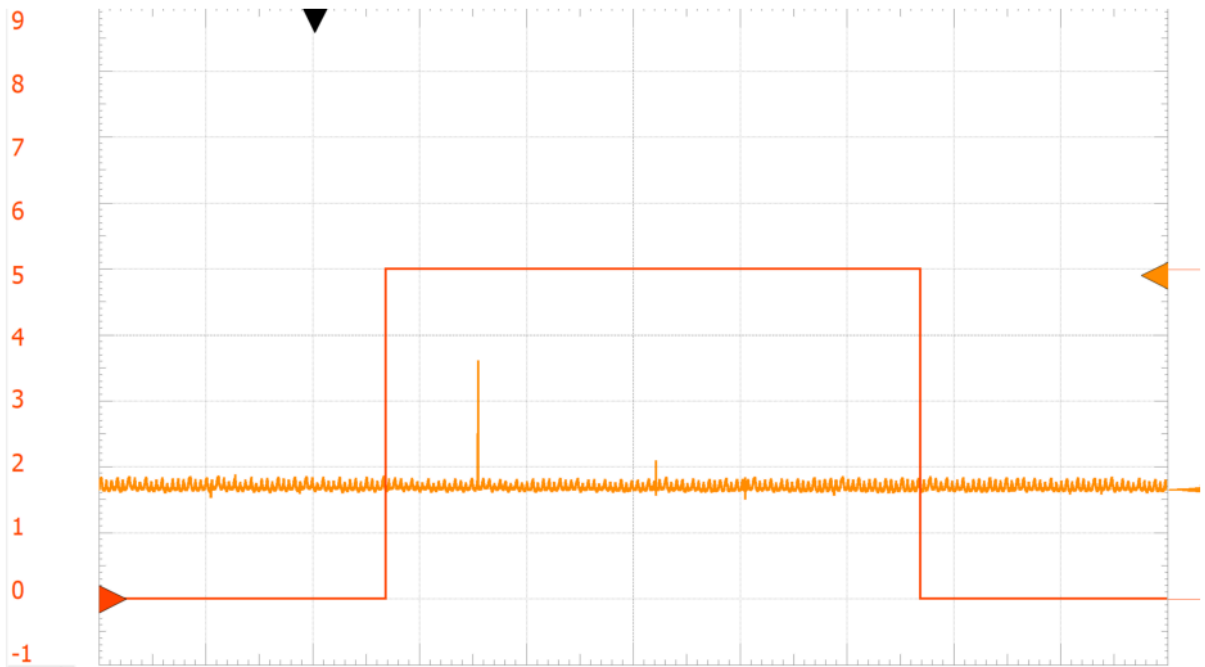


Figure 34, OpAmp output (orange) with 5Hz reset pulse, 500Hz CLK.

It is super noisy. I asked Prof. about it, and it turns out it is simply noise from my motor. I was running the motor with the rest of the circuit, just with CLK disconnected so I could input my own CLK frequency. (I had built the rest of my circuit at this point for sake of time). I unplugged the motor, and the noise went away.

I Here is a screenshot of the different voltages/OpAmp outputs at different input frequencies:

In [62]: `InlineImage('allvolts.png', width=12, height=12, caption= "Figure 35, OpAmp`

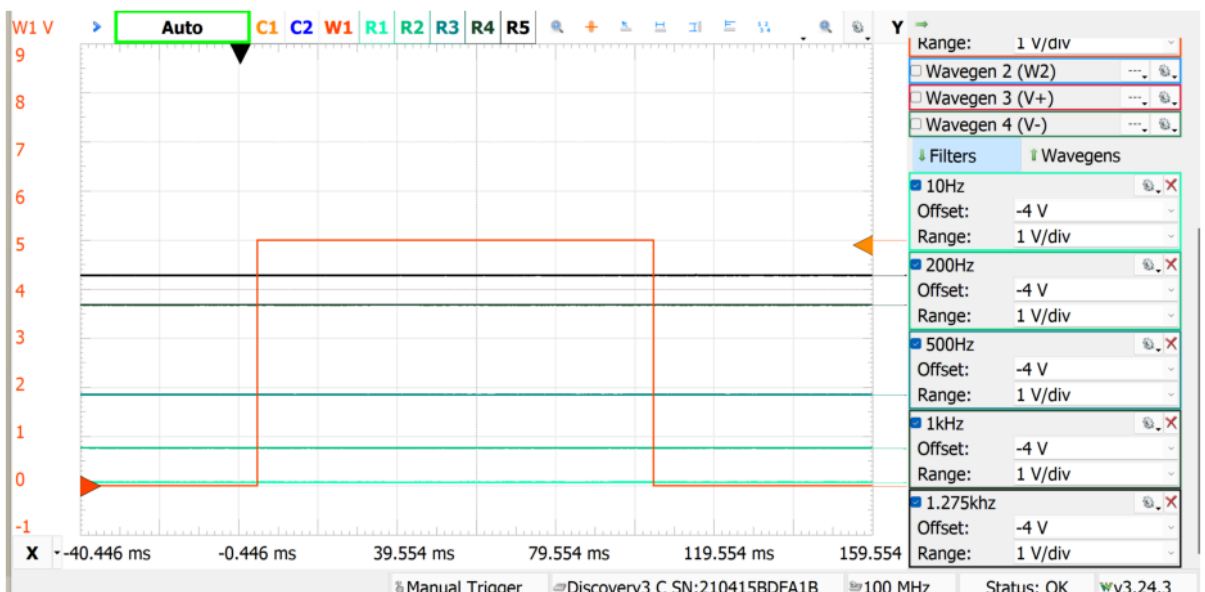


Figure 35, OpAmp outputs of analogue reading for latch value.
CLK frequency of 10, 200, 500, 1000, and 1275 Hz (lightest to darkest)
CLK wave in orange.

A value of 255 is the maximum value on the latch, so I can only go up to 1275 Hz CLK input. (5Hz * 255). If I go past this, the CLK will reset and the voltage reading will go back to near 0. I can change my reset pulse frequency to accomodate for faster motors which will have a higher CLK input frequency. However, I doubt there will be a need for anything too crazy like that in the near future.

##2.3 Task 3, Motor Sensor & motor driver

Built the rest of the circuit. Motor works and I get the expected reading on the OpAmp that corresponds with how fast the motor is spinning. If I unplug the motor, the OpAmp voltage gradually changes to 0.

Here is an image of how I have connected the motor/motor sensor to the breadboard:

```
In [69]: #image of motor as well as circuit diagram of connections here.  
InlineImage('motorcon.png', width=12, height=12, caption= "Figure 36, Motor
```

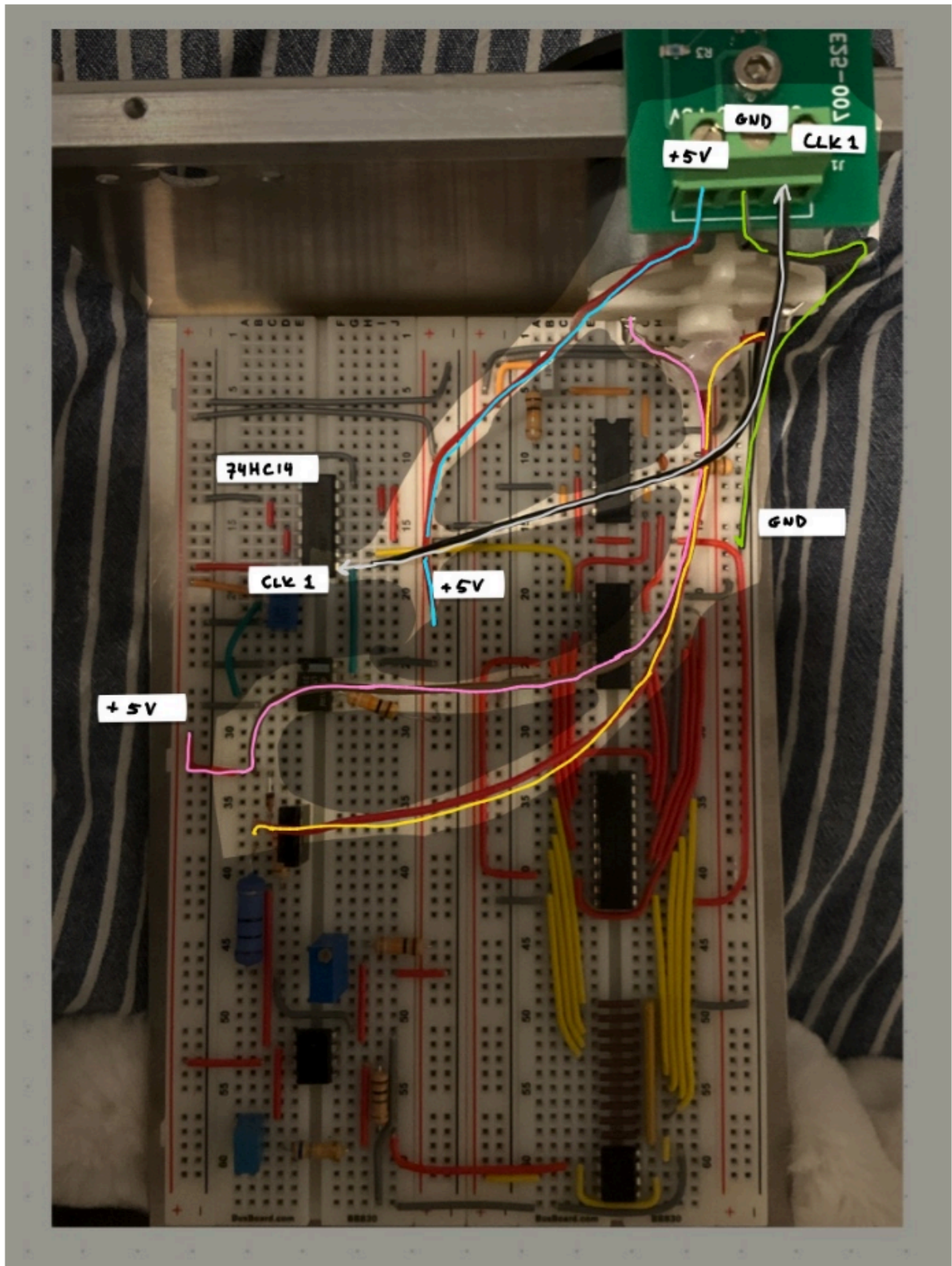



Figure 36, Motor connections to breadboard.

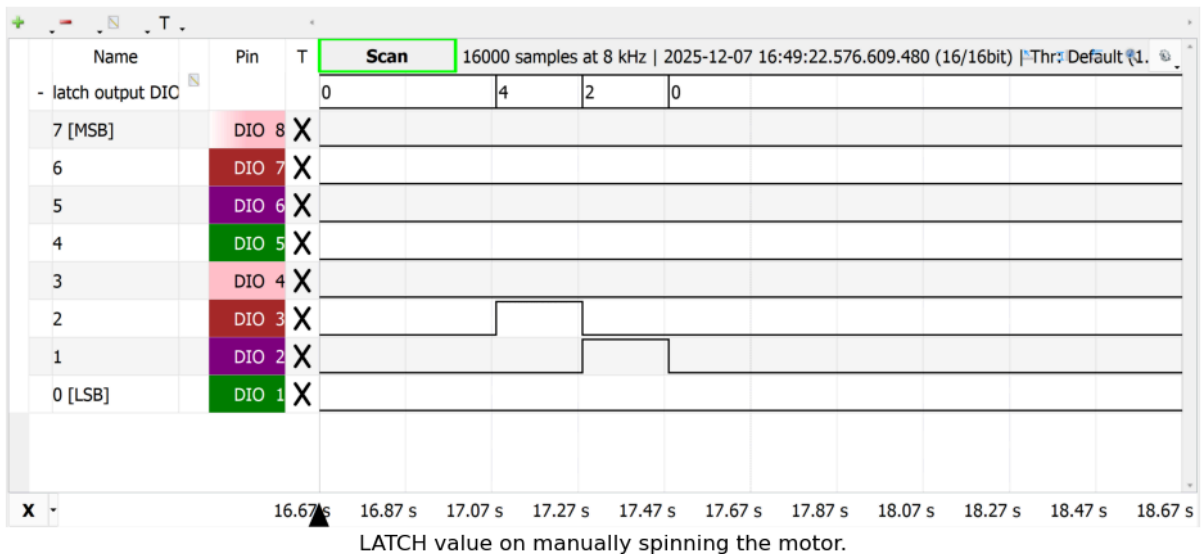
Don't worry! I will have a better labelled image with where the positive and negative leads of the motor itself are connected! This image is more to show the overall progress of my circuit at this point. Additionally, it offers a view of where the leads for the phototransistor part of the circuit connect to. We have the CLK

lead, which transmits a pulse every time a hole passes a sensor. This is fed to an inverter. We also have the Ground and VCC as always.

NOTE the CLK pulse from the phototransistor circuit does not count a full revolution! there are 10 holes, so it counts 1 tenth of a revolution.

Quick image of the motor generating CLK pulses when I gave it a spin:

In [92]: `InlineImage('manualturn.png', width=12, height=12, caption= "LATCH value on`



##2.4 Task 4, Error Signal Amplifier

For the error signal amplifier, I used the following parts:

- 1x UA741 OpAmp
- 1x 1M resistor
- 1x 680k resistor
- 1x 10k resistor
- 1x 10k potentiometer
- 1x 10 ohm 3W resistor
- 1x 22k resistor
- 1x MJF122 Transistor
- 1x 1N4148 diode
- 1x 1uF capacitor
- the motor kit.

This is the diagram of the circuit from the lab manual:

In [70]: `InlineImage('servocirc.png', width=12, height=12, caption= "Figure 37, circu`

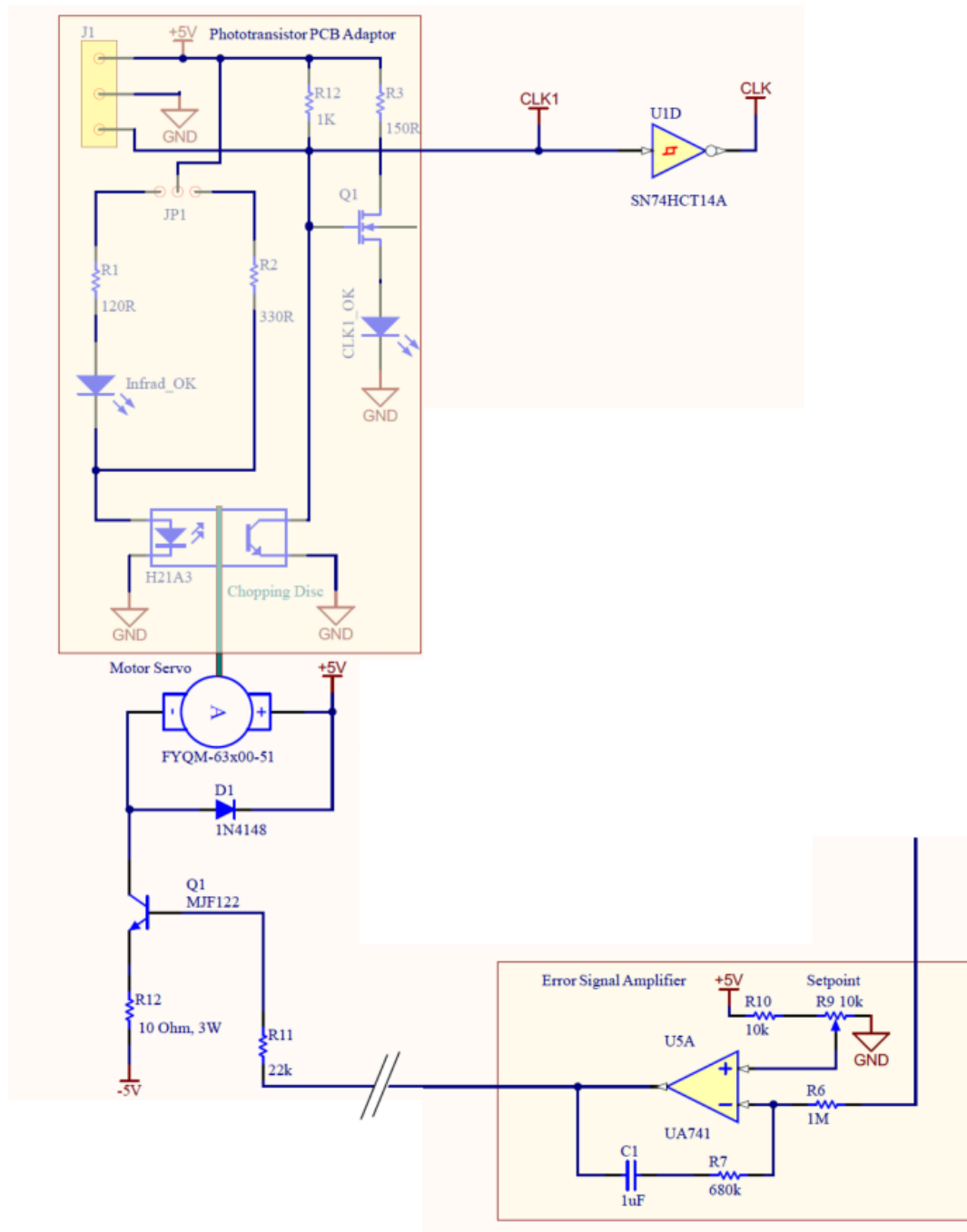


Figure 37, circuit diagram of servo portion of the circuit.

Here are the datasheets/pinout diagrams for the new parts used in this section:

In [74]: `InlineImage('npndata.png', width=12, height=12, caption= "Figure 38, NPN tra`

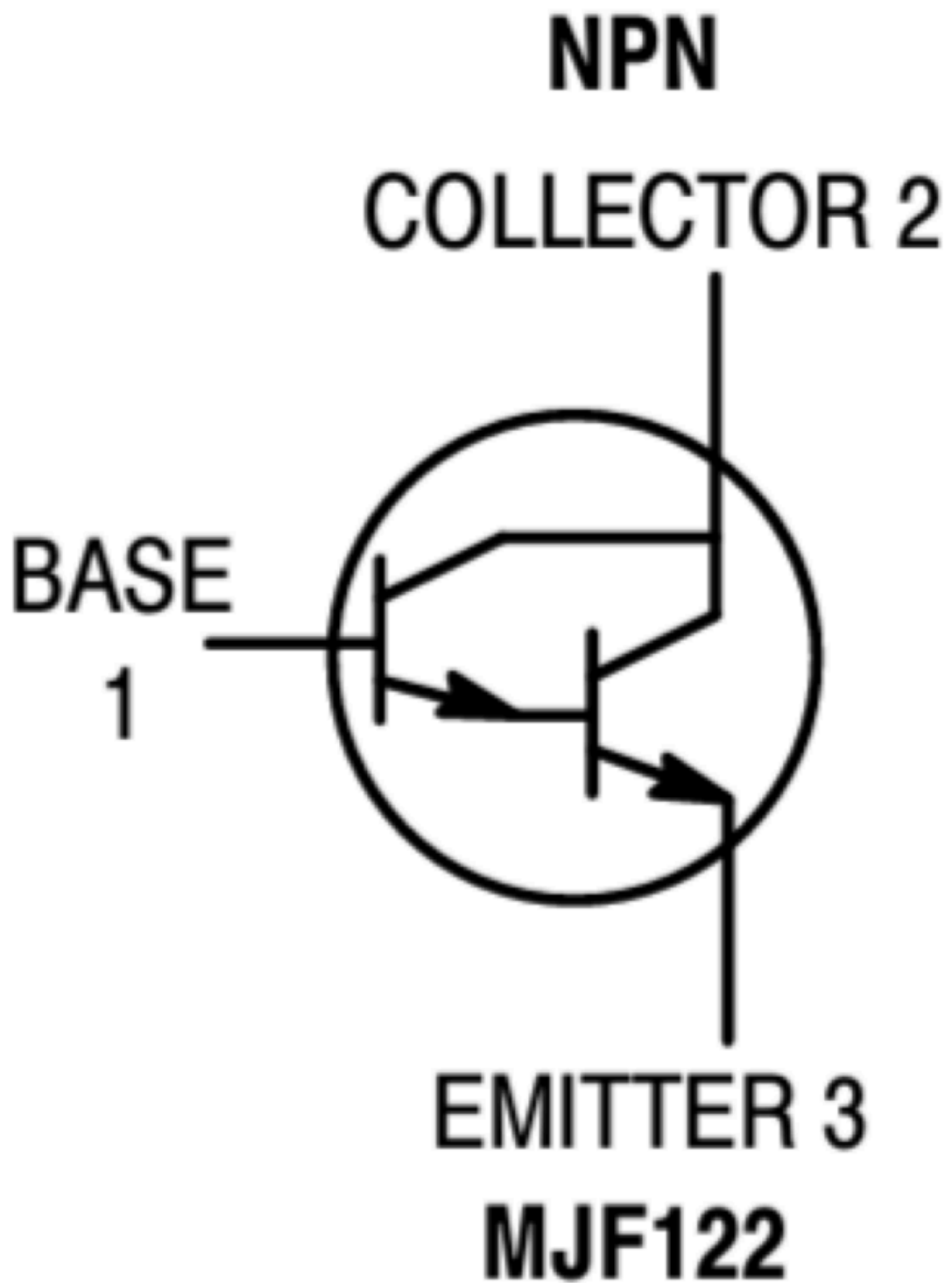


Figure 38, NPN transistor pinout diagram.

Here are my circuit diagrams, as well as annotated circuit photos:

```
In [73]: #photos and diagrams for error amplifier here.  
InlineImage('errordiag.png', width=12, height=12, caption= "Figure 39, diagr
```

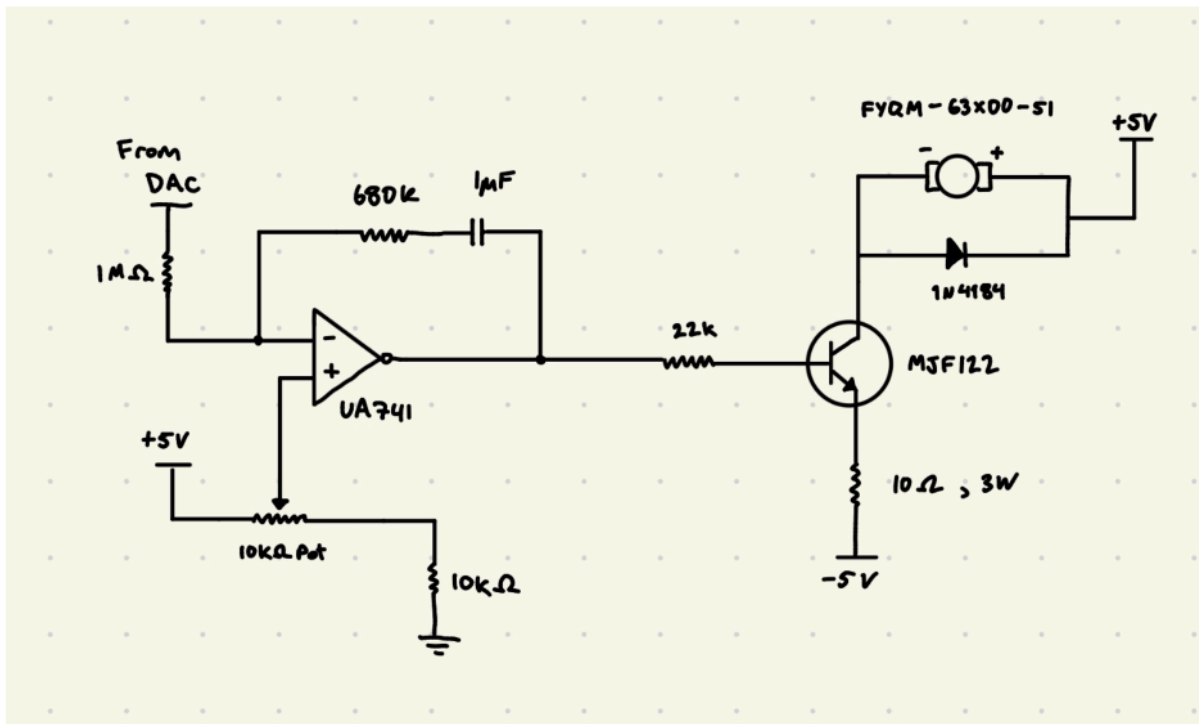


Figure 39, diagram of my error signal amplifier circuit.

In [72]: `InlineImage('errorpic.png', width=12, height=12, caption= "Figure 40, picture of my error signal amplifier circuit.")`

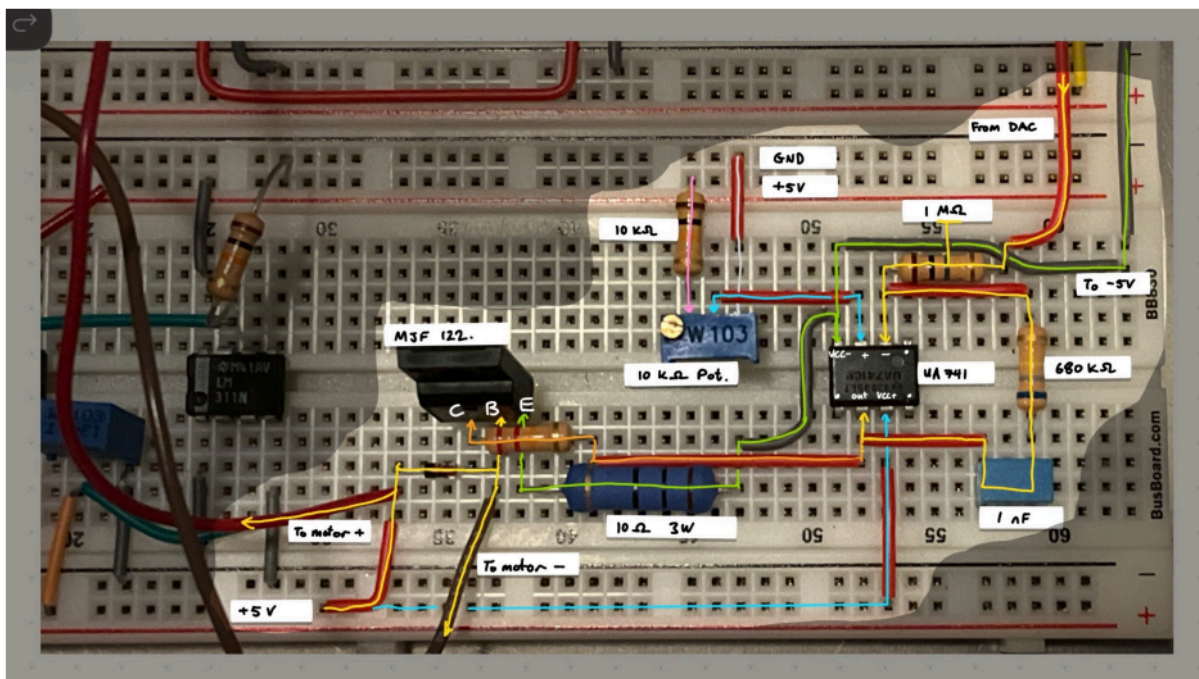


Figure 40, picture of my error signal amplifier circuit.

TS

I have run into an error:

In [75]: `InlineImage('error.png', width=12, height=12, caption= "Figure 41, power supply circuit.")`

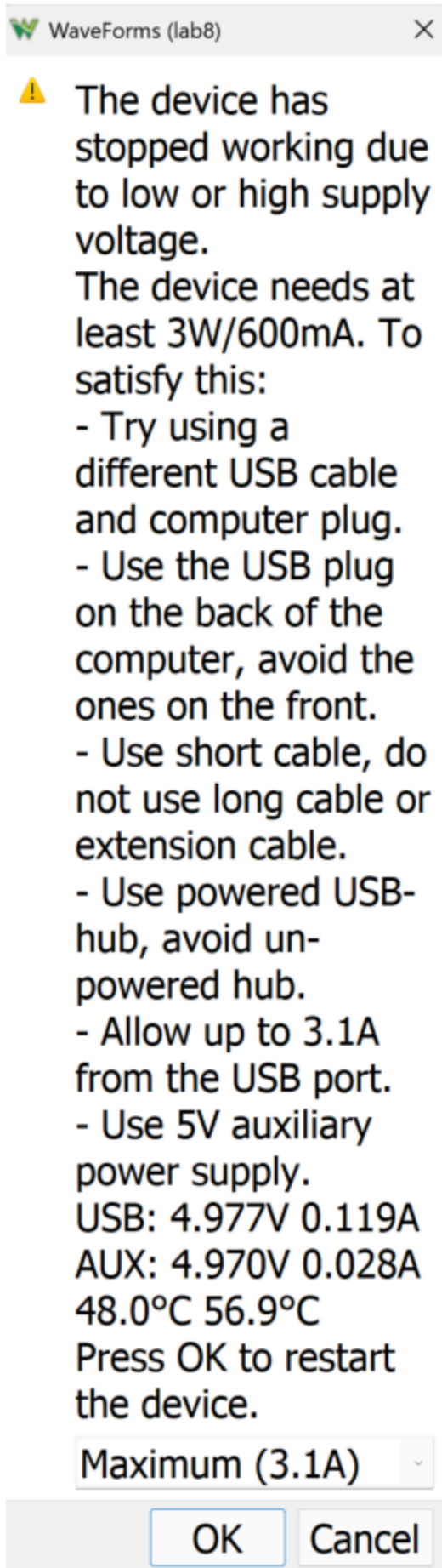


Figure 41, power supply error.

When I run the circuit without an external power supply, I get a very small RPM on the motor, and the voltage sources V_+ and V_- only go to around 2V in magnitude. So I tried using the external power supply in my lab kit. However, I get the above error when I try turning the power sources on again.

It seems something is drawing a lot of power, or I have a short in my circuit somewhere. Here is what I tried:

- Using different power sockets for external power supply. This did nothing.
- Restart waveforms software. This also did nothing.
- I only have 1 usbC port on my laptop, so I couldn't use another usbC port.
- So maybe it is something to do with a short circuit somewhere. The TA said it was unreasonable that any part other than the motor is overdrawing power/current, so I disconnected the motor first.

Inspecting my circuit connections, I found that my diode that runs after the MOSFET in the motor portion of the circuit was backwards. That explains a lot. I had no current at all going to the positive lead of the motor. Problem solved. I re-oriented the diode (I swear I checked it the first time around haha), and now my power source behaves normally.

Ok, now that my connections are stable, I will test the functionality of the error signal amplifier.

I started by deriving an expression for V_{out} as a function of V_{in} for the OpAmp:

```
In [76]: InlineImage('opinout.png', width=12, height=12, caption= "Figure 42, Derivat
```

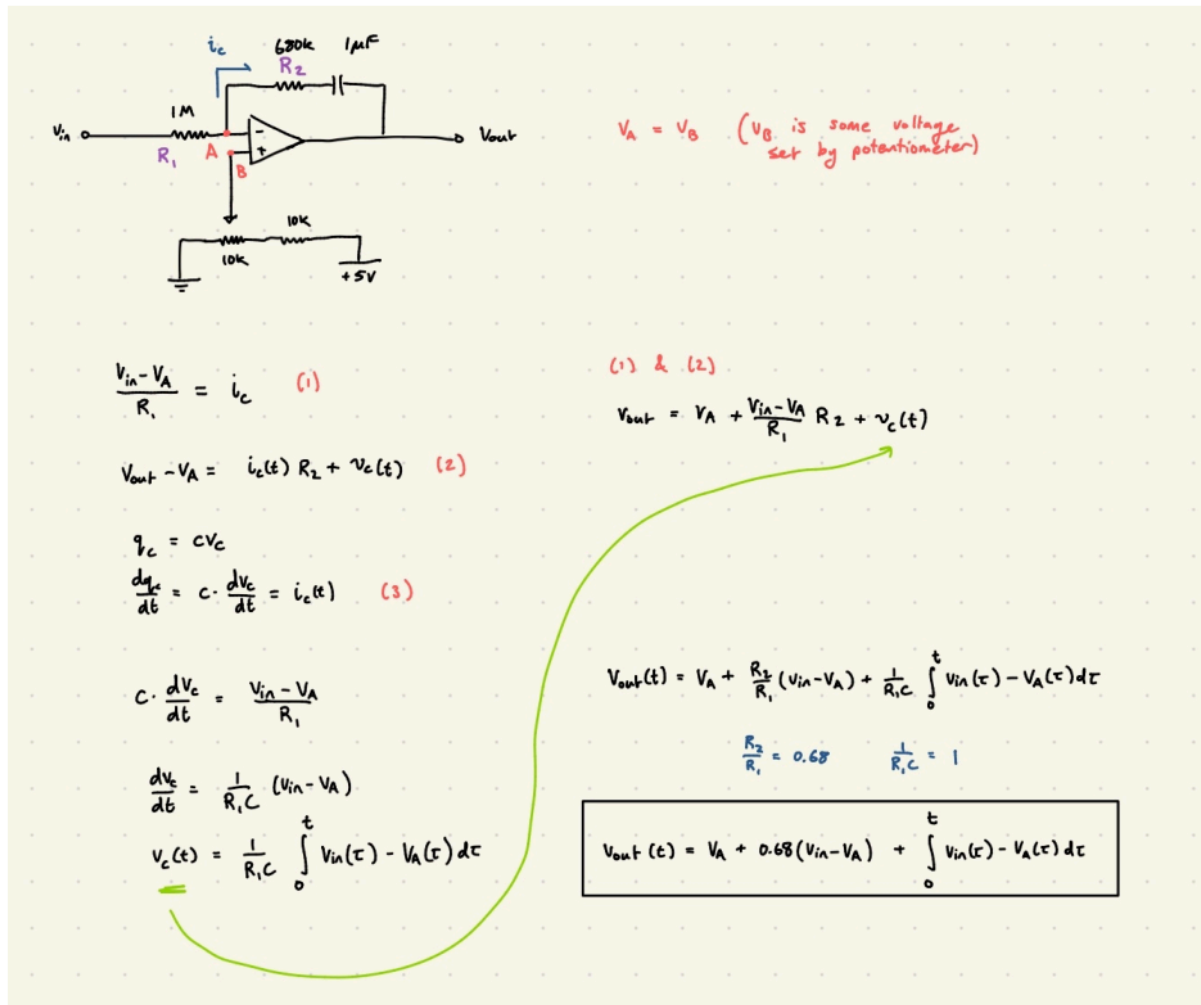


Figure 42, Derivation of V_{out} vs V_{in} for the OpAmp of the error amplifier via nodal analysis.

Essentially, there are a couple terms. First, there is the baseline voltage over the potentiometer. This kinda acts as an offset over the entire waveform.

Then, there is the term that would be an offset, however rather than being constant, this depends on the difference between the potentiometer voltage and the input voltage. This is also multiplied by a factor of 0.68, which I assume is equivalent to a gain value in our case. In the waveform, this would show up as a vertical jump or maybe like a staircase if there is fluctuation in the input.

Finally, there is the integrating term which essentially integrates whatever error is left over after the second offset term. In the waveform, this would show up as a ramp, which is a result of this being essentially a sum.

I am interpreting the first term as like a baseline, the second term as a 'course adjustment', and the last integrating term as a 'fine adjustment' for error.

Basically, this is how the error amplifier works as I understand it:

- the motor spins a given amount of times between reset pulses. these values are fed to the counter/latch/dac, which are converted to a stream of

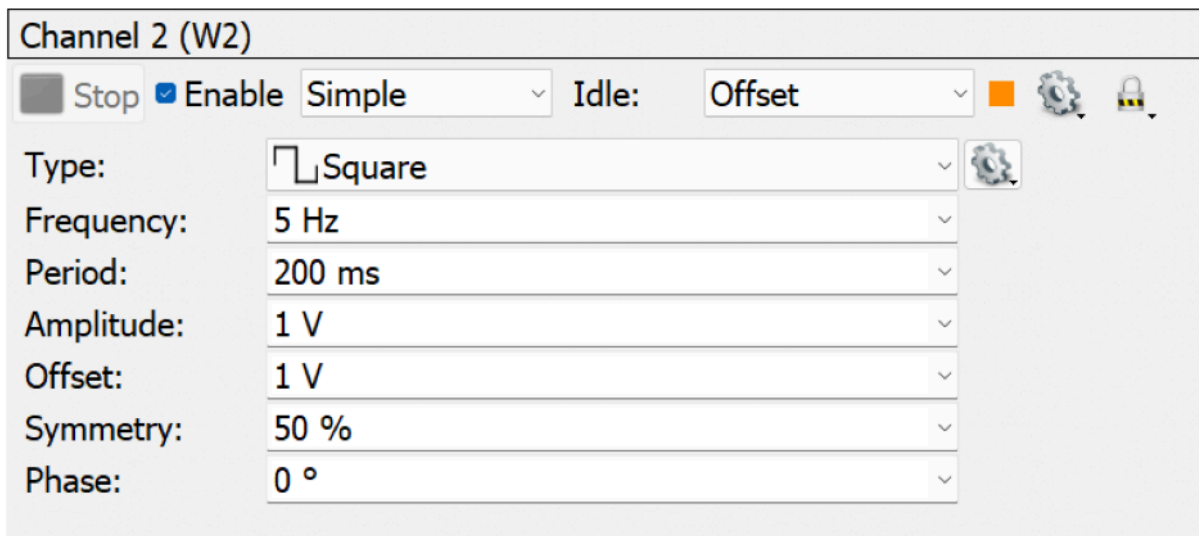
voltage readings which are proportional to the rpm of the motor.

- We want to use the potentiometer to control the speed of the motor.
- the difference between the voltage from the dac and the potentiometer is the 'error'.

With these, we use the error amplifier to gradually coerce the input voltage to match the potetiometer voltage via feedback (by adjusting the output and thus motor speed). The job is to ensure that the motor runs at pretty much a stable rpm, as well as to make it run at a set rpm depending on what we set the potentiometer voltage to be.

To test this, I ran a waveform into the input of the OpAmp with these settings:

```
In [77]: InlineImage('opampinput.png', width=12, height=12, caption= "Figure 43, sett
```



Channel 2 (W2)	
Stop	☒ Enable
Simple	Idle: Offset
Type:	Square
Frequency:	5 Hz
Period:	200 ms
Amplitude:	1 V
Offset:	1 V
Symmetry:	50 %
Phase:	0 °

Figure 43, settings of the input squarewave to test error amp op amp functionality.

Here is what I see when I scoped over my input and output:

```
In [78]: InlineImage('stable.png', width=12, height=12, caption= "Figure 44, stablize
```

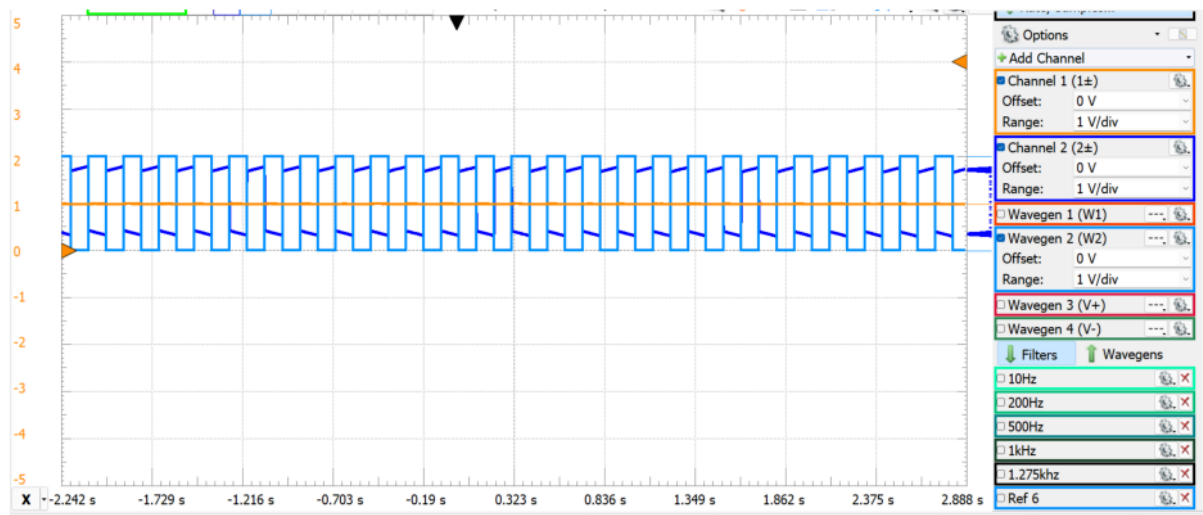


Figure 44, stablized OpAmp output (dark blue), wavegen input (light blue), and potentiometer voltage (orange).

As is evident, this matches what we should expect. there is the sudden jump due to the error term with a gain..? of 0.68, then the ramping due to the integrating term!

A few things to note: I managed to stabilize this waveform without it drifting to either rail and reading a saturated signal, which took some tweaking of the of the potentiometer. I found that the potentiometer value is in the middle of the min/max for the input wave, and this seems to be when the output is most stable (ie not running towards either rail of the opAmp). I think this is likely due to the way the error works; over time, if the average of the input is the same as the potentiometer value, the circuit will correct the error by roughly the same amount (0.68 of the amplitude I am guessing), but in both directions. So when it flip flops, there is actually no considerable change over time.

I also noticed that letting it sit for a few minutes, there was eventually enough error accumulated to push the signal to the -5V rail.

The reason why it drifts is also due to the way it integrates the error, I think. Every time I observe it to drift, I don't see big changes due to the offset term with the 0.68 multiplier. I can only guess that when the voltage over the potentiometer is lower than the average of the input, the output will keep decreasing and decreasing in an attempt to correct this. and vice versa when the voltage over the potentiometer is higher than the average of the input.

However, since I have the motor disconnected at this point for testing, the feedback loop is not yet a loop! so the circuit will keep trying and trying to change the output in hopes of changing the input via feedback, but it cannot until I complete the feedback loop by connecting the full circuit together. I am essentially holding the input constant without letting the error amp correct it despite its best efforts. Thats why the signal drifts to either rail, depending on my setpoint voltage.

Anyhow, I can do that now since I have finally verified everything behaves as I would expect :D

Motor testing

Once I completed the circuit and wired the motor up, I ran into a few issues:

- The latch value fluctuated significantly and would not stay at a certain value.
- the motor sounded like it was spinning at constant speed, but the latch value would still fluctuate greatly; It seemed like random values too, from 10-200+. Seemingly no correlation.
- the DAC output would not converge to the value of the potentiometer contrary to what is expected (the purpose of the error amplifier is to 'push' the output of the dac, towards the voltage value of the potentiometer value).
- turning the potentiometer does next to nothing in regards of the motor speed.

Here is what I see:

```
In [82]: InlineImage('badopamp.png', width=12, height=12, caption= "Figure 45, OpAmp
```

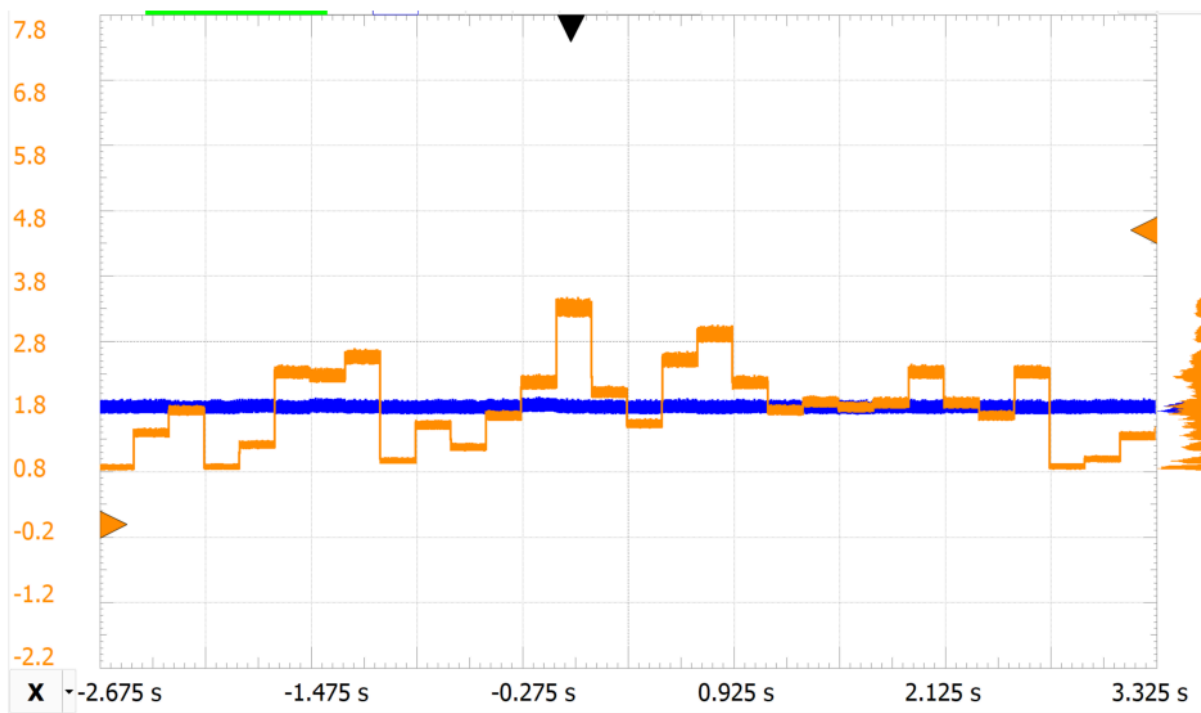


Figure 45, OpAmp input (orange) and setpoint voltage (blue).

```
In [83]: InlineImage('badlatch.png', width=12, height=12, caption= "Figure 46, Latch
```

Name	Pin	T	Scan	16000 samples at 3.2 kHz 2025-12-07 13:53:44.091.803.580 (16/16bit) Thr: Default (																						
- latch output DIC				57	115	83	38	97	99	71	121	131	136	224	200	209	196	254	85	23	15	25	36	126	53	91

Figure 46, Latch value fluctuations. supposed to converge.

Obviously, neither my latch value nor OpAmp input converges to a value. The Error signal amplifier is meant to push the input towards the setpoint voltage, thus regulating the motor speed. That is clearly not happening, and its reflectedd in my latch value as well. If I were getting the correct stabilization, the latch would be a constant value (maybe with some fluctuation +/- 1 or 2).

TS

I am suspecting that there is something wrong with the latch..? maybe something is causing certain bits to flip flop when they should not. This will change the input value to something it actually is not, and affect how the error amplifier attempts to stabilize the motor speed. I quickly verified the latch works again by isolating that part of the circuit, which it does.

Eventually, I asked a TA (Greg) and he told me to scope over the actual CLK output of the phototransistor on the motor apparatus. I did so, and we found that the phototransistor circuit was not registering every single time a hole passed by; Essentially, I was getting a large signal for maybe half the holes, and a small signal for another portion, and sometimes even no signal at all. I unfortunately did not get a screenshot, as Greg immediately was able to help me build something to solve this issue.

As a result, we had to build a comparator circuit and insert it between the CLK from the motor. The part used is an LM311 comparator. Essentially, it compares 2 inputs, and produces a digital output of either HIGH or LOW. Nothing in between. In my case those values are either 0v or 5v.

I have one input, which is the CLK from the phototransistor circuit. I have another input, which is from a setpoint connected to +5V. The comparator compares these 2 inputs, and if CLK is higher than the setpoint even by a little, it outputs HIGH. Vice versa for when CLK is lower than setpoint. I can adjust the sensitivity/threshold value that triggers a CLK pulse by adjusting the setpoint. Basically this solves my issue of when the CLK pulses are too low to be read properly by my counter.

It guarantees that the signal gets read as a pulse, even if the reading from the phototransistor circuit is a small bump.

Now that the comparator has been set up, my circuit works. I can control the speed of the motor with the potentiometer in the error amplifier portion of the circuit, and actually have the motor respond.

Here is a bit of wiring information of the LM311 comparator, as well as some diagrams/photos of how I've got it set up in my circuit.

```
In [87]: #circuit diagram/photo of comparator here.  
InlineImage('comparatorpin.png', width=12, height=12, caption= "Figure 47, L  
InlineImage('compare.png', width=12, height=12, caption= "Figure 48, photo  
InlineImage('comparee.png', width=12, height=12, caption= "Figure , circuit
```

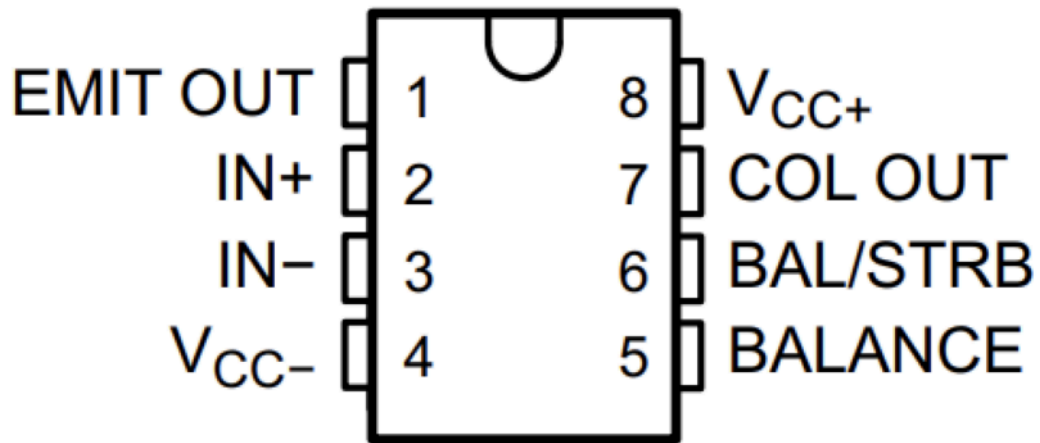


Figure 47, LM311 comparator pinout diagram.

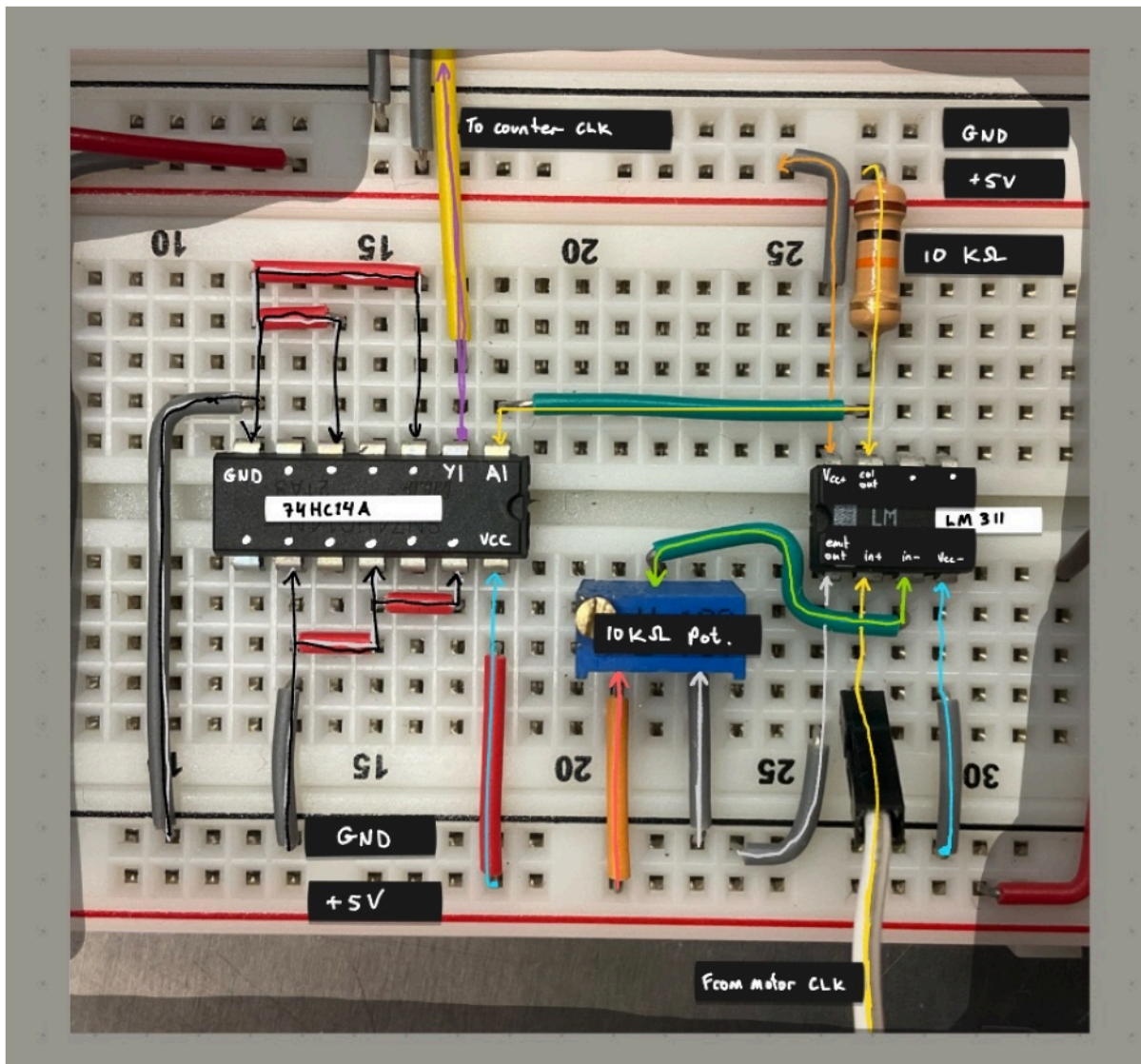


Figure 48, photo of comparator and inverter for CLK pulse.

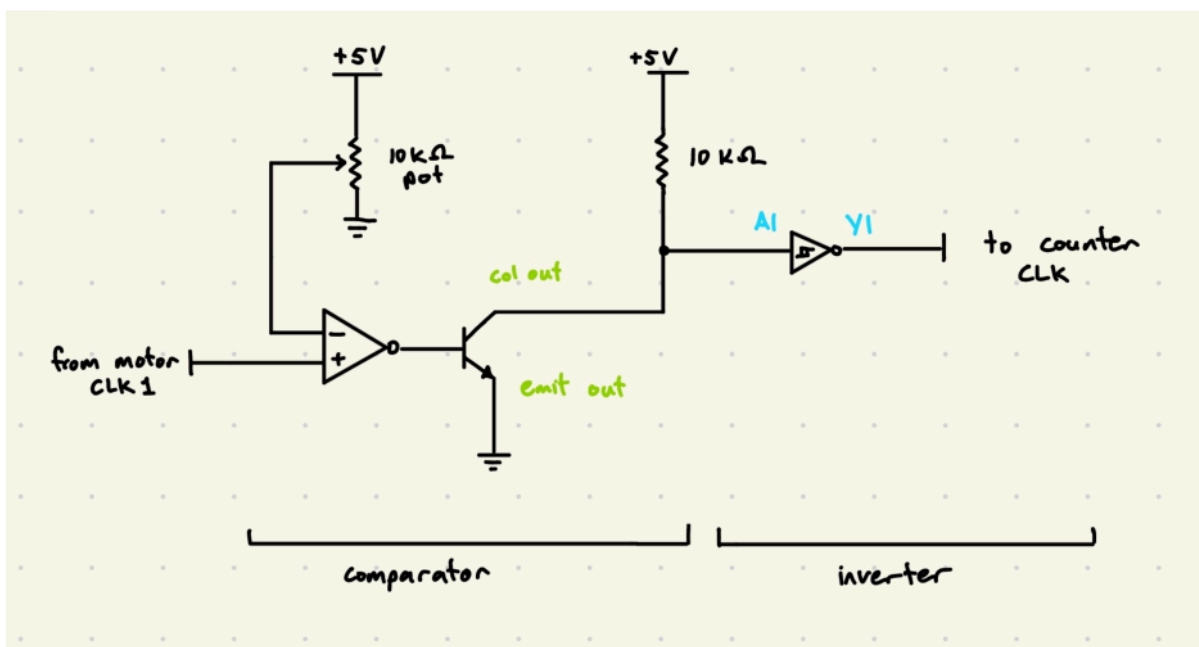


Figure , circuit diagram of comparator and inverter for CLK pulse.

Here are some screenshots of what the comparator does, as well as what the latch displays after the addition of a comparator. Note, there is minor fluctuation of a few digits, but no jumping around to completely uncorrelated values anymore. But anyways it works as intended now, so I'm happy.

```
In [84]: #INSERT IMAGE HERE
InlineImage('compared.png', width=12, height=12, caption= "Figure 50, Compar
```

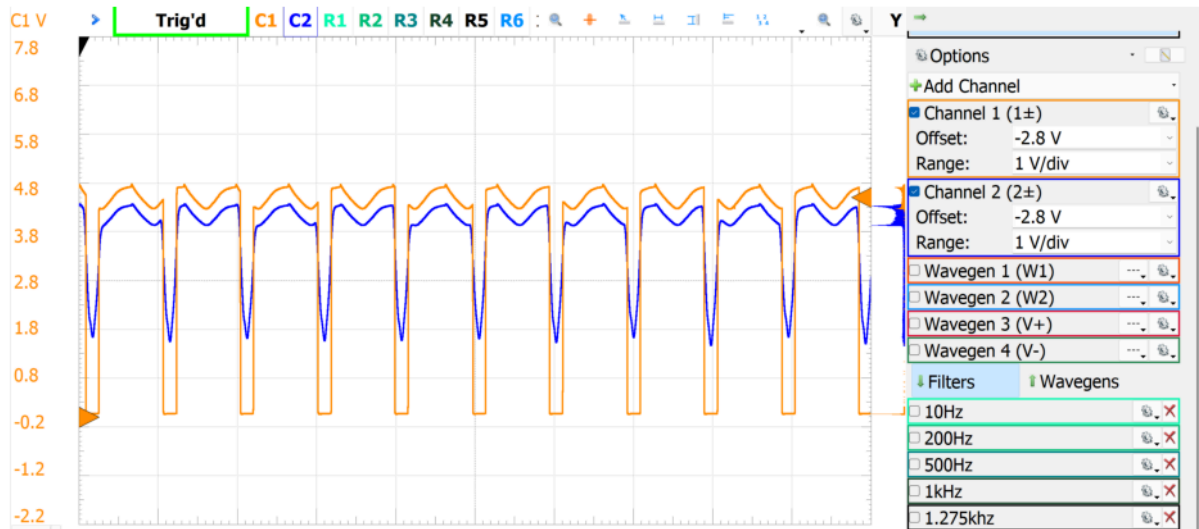


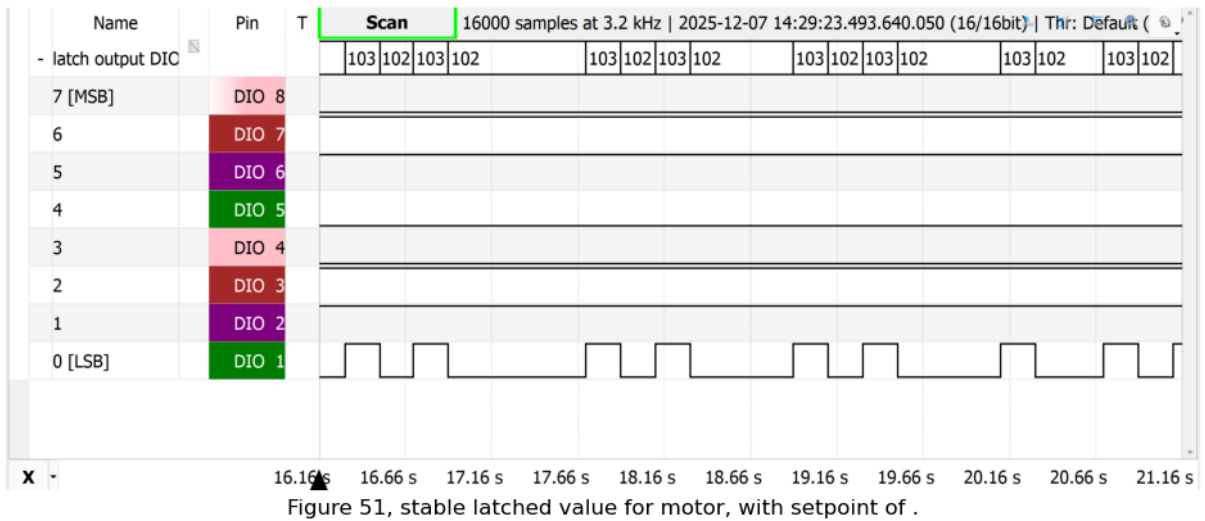
Figure 50, Comparator output (orange) and CLK output (blue).

I took this screenshot the day after the wire pull. the CLK seems to behave a bit better today, but it just doesn't reach the full amplitude that the comparator makes it reach. However, this seems to be the difference as when I short the comparator with CLK, I get the same behaviour as I was getting the previous day.

Anyways, you can clearly see that the comparator in orange pushes it all the way down, making it essentially act as a digital signal. This signal is then inverted so each dip is a clock pulse (HIGH) and fed into the counter!

Now, my motor spins at a stable rate:

```
In [85]: InlineImage('stablelatch.png', width=12, height=12, caption= "Figure 51, sta
```

The setpoint is around 1.9V, so the expected latch value is around $1.9V/4.7V \times 255 = 103.1$. This matches, and holds stable!

I tested this with several other setpoints; as I increase the setpoint voltage, the motor spins higher! vice versa for when I decrease it. The input to the OpAMP (output from DAC) tracks the setpoint voltage.

Quick note from wire pull:

Partner: Pranay O.

We had our 47pF capacitor in the reset generator cut at one of the legs. We noticed that the BUS count was not resetting, so we initially thought the problem lay with the latch. We disconnected the CLK from the motor, and drove it with a 20Hz CLK pulse so we would expect a value of 4 on the BUS. However, we noticed that now it was counting in intervals of 4. We then connected the BUS to the counter, and found the counter works fine; 0-255. So we decided it was either with the reset pulse, or the latch.

We checked the latch first, and verified connections. Seemed fine. bits 1 and 2 weren't flip flopping when putting BUS over the latch, so we checked if any of those bits had been grounded. No.

We then checked the reset pulse generator, scoping over the input and output. No reset pulse was being output! so we scoped over multiple sections of the circuit, eventually finding that the portion after the 47pF capacitor was always at 0V. so that's how we found our issue.

##3.1 Conclusion

In this lab, we put together all the parts we experimented with previously into a full circuit to modulate the speed of rotation for a servo motor.

Key takeaways:

- start simple with troubleshooting. For example, the issue I thought I had with the latch, which turned out to be due to my waveform voltage amplitude.
- There are many possible sources of noise in the circuit, and so having things running while not needed can disrupt measurements.
- For more complicated circuits, troubleshoot one section at a time. My gut intuition tells me to build the whole circuit first, then figure out what's wrong after. However, following the guidance of the lab writeup forces me to make sure every part works before moving onto the next. This saves time troubleshooting, as it allows me to fall back down to the 'baseline' of knowing everything prior to what I just built, works.
- Trust myself more in wiring! It turns out I spent so long double checking connections that troubleshooting could've been a lot quicker. I never ended up having to switch any wires around apart from the diode being backwards.

Here is an overall picture of my circuit!

In [88]: `InlineImage('overview.png', width=12, height=12, caption= "Figure 52, complete circuit overview, sectioned parts.")`

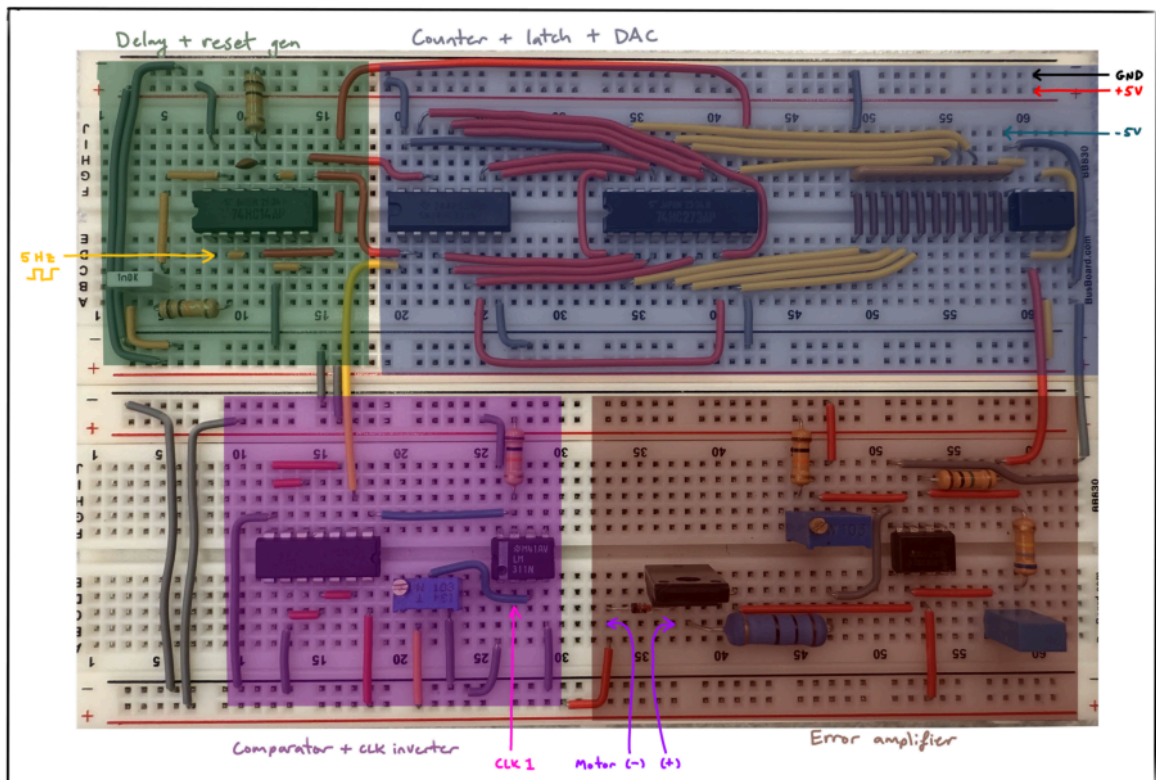


Figure 52, complete circuit overview, sectioned parts.
See respective sections in writeup for circuit diagrams.

Here is my calculation for converting the latch value to RPM.

$$\text{RPM} = \frac{1 \text{ revolution}}{10 \text{ holes}} * \frac{n \text{ holes}}{(latch \text{ value})} * \frac{1 \text{ reset interval}}{1 \text{ reset interval}} * \frac{1 \text{ reset interval}}{0.2s} * \frac{60s}{1 \text{ min}} = 30n$$

where n is the value the latch displays.

so RPM is just the latch number multiplied by 30.

Here is a table of latch values, RPM, and setpoint voltage. I found that around 0.5V is the minimum voltage for the motor to spin, while the RPM gets maxxed out at ~2.25V and stops changing.

In [89]: `InlineImage('datatable.png', width=12, height=12, caption= "Figure 53, conve`

SETPOINT (V)	LATCH VALUE (n)	RPM (rev/min)
0	0	0
0.511	29	870
1.022	57	1710
1.507	86	2580
1.978	112	3360
2.245	133	3990

Figure 53, conversion table of setpoint voltage to latch value and RPM.

Note the max RPM of this motor is approx 3990 RPM, which is around 66.5 rev/second

In [90]: `InlineImage('rpmgraph.png', width=12, height=12, caption= "Figure 54, plotte`

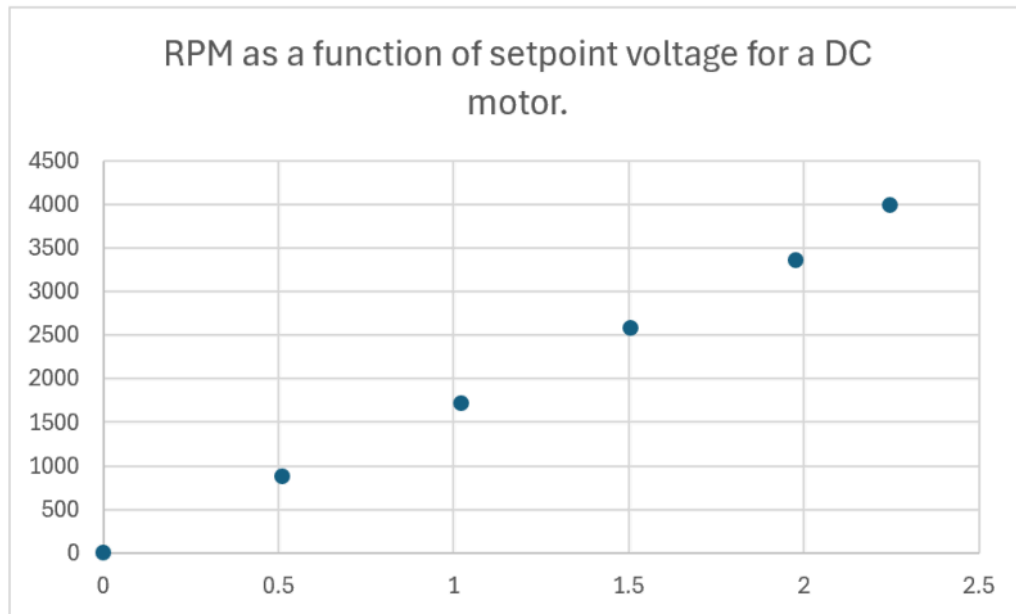


Figure 54, plotted RPM vs setpoint V.

Generally linear relationship, which is to be expected.

```
In [5]: # comments in Python start with a "#" and will not be executed - well commen
# Importing modules that will be necessary for the tutorial
import numpy as np # numpy is a general math/array manipulation library that
import matplotlib.image as mimg # matplotlib.image is used for displaying
import matplotlib.pyplot as plt # matplotlib.pyplot is used for general plot
import pandas as pd # pandas is a data manipulation library, we will use it
print('All necessary modules imported!')
```

All necessary modules imported!

```
In [6]: def InlineImage(img, width=8, height=8, caption=None, fs=24):
# Author: BG
# Renders an image with a caption underneath using matplotlib
# Requires matplotlib.pyplot to be imported as plt and matplotlib.image to
# img - string, the location of an image file
# width, height - numbers, the dimensions of the figure the image is dis
# caption - string, the caption to be displayed beneath the image
# fs - number, the font size of the caption
fig,ax = plt.subplots(figsize=(width,height))
ax.imshow(mimg.imread(img))
ax.set_xlabel(caption,fontsize=fs)
ax.tick_params(
    which='both',
    bottom=False,
    top=False,
    left=False,
    labelleft=False,
    labelbottom=False)
ax.spines["top"].set_visible(False)
```

```
ax.spines["left"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["bottom"].set_visible(False)
```

```
In [ ]: InlineImage('.png', width=12, height=12, caption= "Figure , .", fs=12);
```